Software Architecture Document

Supply Chain Management Simulator

Field	Value
Project	Supply Chain Management Simulator
Version	1.1
Authors	Team 02 – Rohith Karning, Rohanth Ganesha C, Rishi J Mule, Pranav Achar
Date	09-11-2026
Status	Final (pending approval)

Revision History

Version	Date	Author(s)	Description of change	Approval
1.1	09-11-2026	Team 02	Updated for Sprint 1 and 2 changes and final documentation audit (see `docs/deviations.md`)	Pending
1.0	11-10-2026	Team 02	Initial draft	Pending

Approvals

Name	Role	Signature	Date
	Course Coordinator		
Rohith Karning	Product Owner / Integrator		
Rohanth Ganesha C	QA Lead		
Rishi J Mule	Security Test Engineer		
Pranav Achar	Dev Lead		

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Intended Audience
    - 1.4 Definitions, Acronyms and Abbreviations
2. Document Overview
    - 2.1 How to Use This Document
    - 2.2 Related Documents
3. Architecture
    - 3.1 Architectural Goals and Constraints
    - 3.2 Stakeholders and Concerns
    - 3.3 Component Diagram and Domain Class Diagram
    - 3.4 Component Descriptions
    - 3.5 Architecture Pattern and Rationale
    - 3.6 Technology Stack and Data Stores
    - 3.7 Risks and Mitigations

    - 3.8 Traceability to Requirements
    - 3.9 Security Architecture
4. Design
    - 4.1 Design Overview
    - 4.2 UML Sequence Diagrams
    - 4.3 API Design
    - 4.4 Error Handling and Logging
    - 4.5 UX Design
    - 4.6 Open Issues and Next Steps
5. Appendices
    - 5.1 Glossary
    - 5.2 References
    - 5.3 Tools

---

1. Introduction

1.1 Purpose

This Software Architecture Document (SAD) describes the architecture and detailed design of the **Supply Chain Management Simulator** (Python package `scm_sim`), document version 1.1. It explains how the system is decomposed into components, how those components depend on and interact with each other, which architectural pattern was chosen and why, and how each component's public interface behaves, including its errors.

The SAD is the bridge between the requirements in the Software Requirements Specification (SRS v1.1) and the verification described in the Test Plan (Test Plan v1.1). Every design element is traced back to requirement IDs from `docs/requirements-registry.md` (`SCM-F-xxx`, `SCM-NF-xxx`, `SCM-SR-xxx`).

The architecture described here is the implemented Sprint 2 design, including the deviations in `docs/deviations.md`. Component, class, method, exception and file names are checked against the source in `src/scm_sim/`. This document follows the viewpoint-based approach of ISO/IEC/IEEE 42010:2011, adapted to the course template.

1.2 Scope

The document covers:

- The **static structure** of the system: seven components (Inventory, Order Manager, Shipment Manager, Analytics, Simulation Engine, CLI, CI Pipeline), the domain classes inside them, and their dependencies (§3.3, §3.4).
- The **architecture pattern**, a layered, modular architecture inside one Python package, together with the alternatives that were rejected (§3.5).
- The **dynamic behaviour** of one simulated day and two key scenarios, order fulfilment with back-ordering and detection of a delayed shipment, as UML sequence diagrams (§4.1, §4.2).
- The **API design** of the six code components: signature, parameters, return value, exceptions and an example for every public call (§4.3).
- **Error handling**, the command-line **user experience**, **security architecture** (STRIDE) and **open issues** (§3.9, §4.4–§4.6).

The following are outside the scope of this document, because the system does not have them: graphical or web user interfaces, databases or persistent scenario-state storage, network services, user accounts and authentication. The CLI may export daily metrics to a user-selected CSV file; this does not persist simulation state. The development pipeline (GitHub, pull requests and GitHub Actions CI) is covered only where it supports the quality and security requirements.

1.3 Intended Audience

Reader	Sections of most interest
Course Coordinator / Instructor / Evaluator	§3.3–§3.5 (structure and pattern), §3.8 (traceability), §4.2 (sequence diagrams)
Developers (team members)	§3.4 (component responsibilities), §4.3 (API), §4.4 (error handling), §4.6 (open issues)
Testers / QA Lead	§4.3 (expected behaviour and exceptions to test), §3.8 (requirement-to-component mapping)
Security Test Engineer	§3.9 (STRIDE threat model), §4.4 (validation and invariants)
Supply Chain Analyst	§4.1 (how a simulated day works), §4.5 (CLI usage and output)

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
Component	A cohesive unit of the system with a defined responsibility and public interface; here, one Python module (or the CI workflow)
Domain layer	The components that hold supply-chain state and rules: Inventory, Order Manager, Shipment Manager
Orchestration	Coordinating the domain components in a fixed order; done by the Simulation Engine
Pure function	A function whose result depends only on its arguments and which changes no state; all Analytics functions are pure
Snapshot	An independent copy of all stock levels, `{(warehouse, sku): qty}`, returned by `Inventory.snapshot()`
Daily step	One call to `Simulation.step()`, which simulates exactly one day
Replenishment	A shipment from a supplier to a warehouse, dispatched automatically when an item reaches its reorder point
Invariant	A condition that always holds, e.g. stock is never negative
SKU	Stock Keeping Unit, e.g. `WIDGET`
SCM	Supply Chain Management
Lead time / Reliability	Supplier's base delivery time in days / probability (0.0–1.0) of arriving on the expected day
Stockout / Back-order / Excess	Stock exactly 0 / an order waiting for stock / stock above an item's max level
Fill rate	Fulfilled orders ÷ (total − cancelled orders)
Dataclass	A Python class declared with `@dataclass`, used for plain data records (`Order`, `Shipment`, `Supplier`, `StockPolicy`, `ScheduledOrder`, `DayLog`)
STRIDE	Threat categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
CI	Continuous Integration (GitHub Actions job `test`)

Term	Definition
CLI	Command-Line Interface (`python -m scm_sim`)
PR	Pull Request
UML	Unified Modeling Language
SRS / SAD	Software Requirements Specification / Software Architecture Document

2. Document Overview

This section describes the organization of the Software Architecture Document (SAD) and lists associated project specifications.

2.1 How to Use This Document

This document defines the architectural blueprint, component design, runtime execution model, and interface contracts for the Supply Chain Management Simulator. It serves as the primary technical reference for engineers, evaluators, and system architects: - **System Evaluators & Instructors:** Review Section 3.1 (Goals and Constraints), Section 3.3 (Component and Class Diagrams), Section 3.5 (Architecture Pattern), and Section 4.5 (UX Design) to verify architectural compliance, modularity, and runtime simplicity. - **Developers & Integrators:** Consult Section 3.4 (Component Descriptions), Section 4.1 (Design Overview), Section 4.2 (Sequence Diagrams), and Section 4.3 (API Design) for concrete class interactions, method signatures, parameter types, error handling, and lifecycle semantics. - **QA & Test Engineers:** Reference Section 3.8 (Traceability to Requirements) and Section 4.4 (Error Handling and Invariants) to corroborate architectural boundaries against the corresponding verification suites defined in the Test Plan.

2.2 Related Documents

The architecture documented herein directly realizes the functional requirements, quality attributes, and verification plans defined across the core engineering deliverables: 1. **Software Requirements Specification (SRS) v1.1:** Defines the 22 functional requirements (`SCM-F-001`–`SCM-F-044`), 5 non-functional requirements (`SCM-NF-001`–`SCM-NF-005`), and security requirements (`SCM-SR-001`–`SCM-SR-007`). 2. **Test Plan v1.1:** Establishes the 15 system test cases (`TC-INV-01`–`TC-REP-01`), coverage criteria, entry/exit criteria, and automated test automation mapping. 3. **Requirements Traceability Matrix (RTM) (SRS Section 8):** Maps each requirement ID to its architectural module, design section, and verifying test suite. 4. **Repository Documentation (`README.md`):** Outlines repository structure, execution instructions, CLI usage parameters, and developer setup.

3. Architecture

3.1 Goals and Constraints

The architectural design of the Supply Chain Management Simulator balances rigorous academic software engineering objectives with practical operational constraints.

3.1.1 Architectural Goals

1. **Functional Correctness & Invariant Enforcement:** Ensure supply chain physical realities are strictly upheld. Inventory quantities cannot drop below zero, warehouse storage cannot exceed declared capacity limits, and orders cannot experience partial fulfillment or invalid state transitions. All mutating operations follow fail-fast semantics, guaranteeing state remains completely unchanged if an exception is raised.
2. **Deterministic Reproducibility:** Ensure identical execution outcomes given the same initial scenario configuration and random seed (`SCM-NF-002`). By centralizing all stochastic delay decisions within a single pseudo-random generator (`random.Random(seed)`), simulations produce reproducible audit logs and metrics across platforms.

3. **High Testability & Verification:** Architect all components as modular, decoupled units capable of being tested in isolation without requiring external database drivers, web servers, or network mocks (SCM-NF-003). Testability targets $\geq 90\%$ branch/statement coverage in continuous integration.
4. **Parallel Team Development:** Establish clean, one-way dependency contracts enabling a 4-person engineering team to construct and test domain modules (`inventory.py`, `orders.py`, `shipments.py`, `analytics.py`) concurrently before integration into the simulation orchestrator (`simulator.py`).

3.1.2 Architectural Constraints

1. **Standard Library Runtime Only:** The runtime environment must utilize exclusively the Python 3.11 standard library (`dataclasses`, `enum`, `random`, `argparse`). No third-party packages are permitted at runtime (SCM-NF-004). External libraries are restricted strictly to testing (`pytest`, `pytest-cov`).
2. **In-Memory Volatile Storage:** The simulator operates purely in RAM using in-memory data structures (Python dictionaries and lists). There are no external databases, file persistence engines, or caching daemons.
3. **Command-Line Interface (CLI):** The software interacts with users strictly through a console CLI entry point (`python -m scm_sim`) and programmatic Python APIs. Web graphical user interfaces, microservices, and network protocols are explicitly out of scope.
4. **Academic Semester Timeline:** The project scope is constrained to the academic course schedule, prioritizing robust core domain logic, deterministic simulation stepping, and comprehensive documentation over commercial distributed deployment.

3.2 Stakeholders and Concerns

The Supply Chain Management Simulator addresses the operational, educational, and architectural concerns of four key stakeholder roles:

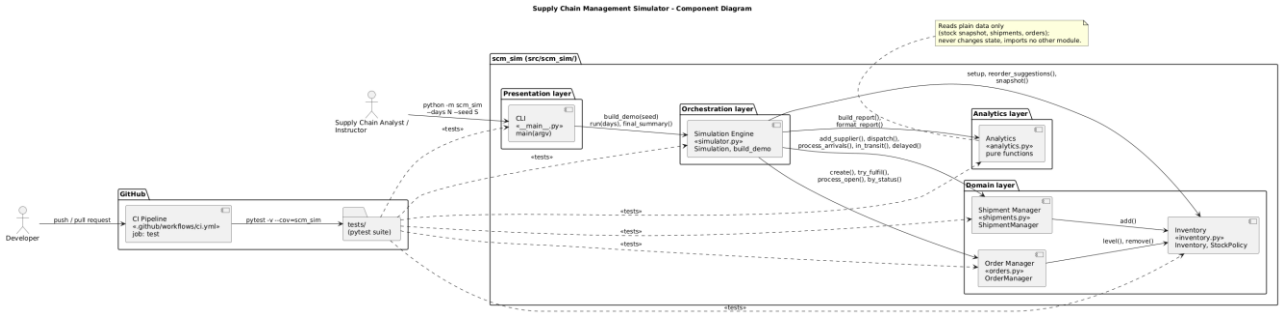
Stakeholder	Role in Project	Primary Architectural Concerns	Architectural Solutions & Mitigations
Supply Chain Analyst	Primary end user configuring scenarios and evaluating operational performance	- High-fidelity detection and reporting of delays, stockouts, back-orders, and excess inventory.- Deterministic simulation reproducibility so that identical configurations yield identical results.- Intuitive, clean command-line reporting without noisy stack traces.	- Analytics module (`scm_sim.analytics`) produces structured report dictionaries and formatted summaries with explicit counts and “No issues” indicators.- Dependency injection of seeded pseudorandom number generators (`random.Random(seed)`) ensures run-to-run determinism.- Clear CLI output streaming daily logs and final multi-line summary.
Course Instructor / Evaluator	Academic evaluator assessing software quality, compliance, and functionality	- Ability to execute demo scenarios with a single CLI command.- Thorough, automated test coverage (>= 90%) validating all requirements.- Clean software engineering documentation following IEEE standards.	- Module execution entry point (`python -m scm_sim --days 10 --seed 42`) runs pre-configured realistic scenario (`build_demo()`) in < 1 second.- Comprehensive pytest test suite (100 automated tests, 99.79%

Stakeholder	Role in Project	Primary Architectural Concerns	Architectural Solutions & Mitigations
			line coverage) executed by GitHub Actions on pull requests and pushes to `main`.- Traceable SRS, SAD, and Test Plan mapped to requirements and automated test cases.
Module Developers (Team Members)	Software engineers building and maintaining subsystem modules in parallel	- Strong module decoupling and encapsulation to prevent cross-module interference.- Clear, documented public APIs with well-defined parameter types and exception guarantees.- Fast, reliable local unit test execution without external services or network setup.	- Layered modular architecture: each developer owns an isolated module (`inventory.py`, `orders.py`, `shipments.py`, `analytics.py`).- Strict in-memory operation using only the Python standard library and pytest.- Invariant enforcement: operations fail fast and leave state completely unchanged on error.
Team Lead / Integration Owner	Integrator combining modules into cohesive simulation orchestrator	- Seamless integration of independently developed modules into the daily simulation loop.- Prevention of merge conflicts and regression errors.- Transparent commit history and pull request accountability.	- Simulation facade (`scm_sim.simulator.Simulation`) orchestrates daily execution sequence in a fixed order.- GitHub branch protection on `main` enforcing PR reviews, green CI checks, and linear git history.

3.3 Component Diagram and Domain Class Diagram

This section gives two static views of the system: the **component view**, which shows the seven components and their dependencies, and the **domain class view**, which shows the classes inside the code components with their key fields, methods and relationships. Both diagrams are written in PlantUML; their sources are `docs/diagrams/component-diagram.puml` and `docs/diagrams/class-diagram.puml`.

3.3.1 Component Diagram



Component diagram: Supply Chain Management Simulator

Figure 3.3-1: Component diagram

Components

Component	File	Layer	Owner	Requirements
CLI	`src/scm_sim/__main__.py`	Presentation	Rohith	SCM-F-043, SCM-F-044; supports SCM-F-034, SCM-NF-005
Simulation Engine	`src/scm_sim/simulator.py`	Orchestration	Rohith	SCM-F-040 – SCM-F-042; supports SCM-F-044
Inventory	`src/scm_sim/inventory.py`	Domain	Rohith	SCM-F-001 – SCM-F-004
Order Manager	`src/scm_sim/orders.py`	Domain	Rohanth	SCM-F-010 – SCM-F-013
Shipment Manager	`src/scm_sim/shipments.py`	Domain	Rishi	SCM-F-020 – SCM-F-023
Analytics	`src/scm_sim/analytics.py`	Analytics	Pranav	SCM-F-030 – SCM-F-034
CI Pipeline	`.github/workflows/ci.yml`	Development infrastructure	Rohith	SCM-NF-003, SCM-SR-005

Dependencies (solid arrows: "uses / calls")

From	To	Interface used	Purpose
Analyst / Instructor	CLI	`python -m scm_sim --days N --seed S`	Run the demo
CLI	Simulation Engine	`build_demo(seed)`, `Simulation.run(days)`, `Simulation.logs`, `Simulation.final_summary()`, `metrics_rows(history)`, `describe_day()`	Build the scenario, run it, print the log and report, and prepare daily CSV rows
Simulation Engine	Inventory	`register_warehouse`, `set_policy`, `add`, `reorder_suggestions`, `total_in_warehouse`, `snapshot`	Set-up, replenishment decisions, report input
Simulation Engine	Order Manager	`create`, `try_fulfil`, `process_open`, `by_status`	Serve today's demand, retry back-orders, collect orders for the report
Simulation Engine	Shipment Manager	`add_supplier`, `dispatch`, `process_arrivals`, `in_transit`, `delayed`	Register suppliers, replenish, deliver arrivals, collect late shipments
Simulation Engine	Analytics	`build_report`, `format_report`, `summarize_run`, `format_run_summary`	Build daily reports and aggregate the final run summary
Order Manager	Inventory	`level`, `remove`	Check and take stock when fulfilling

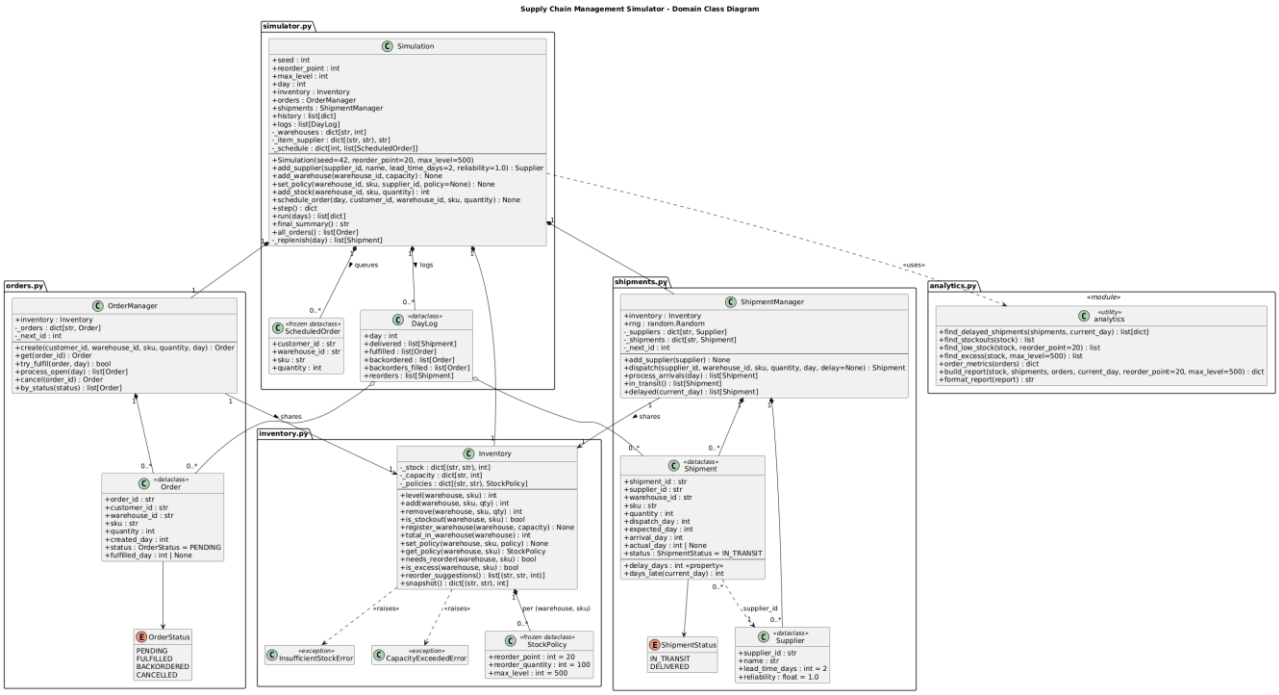
From	To	Interface used	Purpose
Shipment Manager	Inventory	`add`	Put arriving goods into stock
Developer	CI Pipeline	`git push` / pull request	Trigger the `test` job
CI Pipeline	`tests/`	`pytest -v --cov=scm_sim --cov-branch --cov-fail-under=90`	Run the whole suite and enforce the line-coverage gate while measuring branches

Dashed arrows labelled `<<tests>>` show that the pytest suite in `tests/` exercises every code component: `test_inventory.py`, `test_orders.py`, `test_shipments.py` and `test_analytics.py` test their modules on their own, and `test_simulator.py` tests the Simulation Engine and the CLI end to end.

Key properties of the component structure

1. **Dependencies point one way.** The CLI depends on the Simulation Engine, which depends on the domain and analytics components; no component depends on the CLI or the Simulation Engine. This prevents cycles and lets each lower component be tested without the ones above it.
2. **Inventory is the only shared state.** The Order Manager and Shipment Manager receive the *same* `Inventory` object in their constructors. Neither imports the other; they interact only through stock levels, and the Simulation Engine decides when each one runs.
3. **Analytics has no dependencies.** `analytics.py` imports no other `scm_sim` module. It reads plain data (a stock snapshot dictionary and lists of shipment and order objects), so it can be tested with simple stand-in objects and cannot change simulation state.
4. **One place controls event order.** Only `Simulation.step()` calls the domain components during a run, so the order of events within a day is defined in exactly one place (see §4.1).
5. **CI guards `main`.** The CI Pipeline is not part of the runtime product, but it is part of the architecture: the `test` job is a required status check, so code reaches `main` only when the whole suite passes (SCM-NF-003, SCM-SR-005).

3.3.2 Domain Class Diagram



Domain class diagram: Supply Chain Management Simulator

Figure 3.3-2: Domain class diagram (public members +, private members -)

Classes

Class	Module	Kind	Responsibility
`Simulation`	`simulator.py`	Class	Owns one `Inventory`, `OrderManager` and `ShipmentManager`; holds the scenario, the day counter, the report `history` and the `logs`; runs `step()`
`ScheduledOrder`	`simulator.py`	Frozen dataclass	One unit of customer demand queued for a future day (customer, warehouse, SKU, quantity)
`DayLog`	`simulator.py`	Dataclass	Record of one simulated day: shipments delivered, orders fulfilled, orders back-ordered, back-orders filled, reorders dispatched
`Inventory`	`inventory.py`	Class	Stock per (warehouse, SKU), capacity per warehouse, policy per item
`StockPolicy`	`inventory.py`	Frozen dataclass	Reorder point, reorder quantity and max level for one item (defaults 20 / 100 / 500)
`InsufficientStockError`, `CapacityExceededError`	`inventory.py`	Exceptions	Raised by `Inventory.remove` and `Inventory.add` when an operation would break a stock invariant
`OrderManager`	`orders.py`	Class	Creates, fulfils, retries, cancels and lists customer orders
`Order`	`orders.py`	Dataclass	One customer order with its status and fulfilment day
`OrderStatus`	`orders.py`	Enum (`str`)	`PENDING`, `FULFILLED`, `BACKORDERED`, `CANCELLED`
`ShipmentManager`	`shipments.py`	Class	Registers suppliers, dispatches shipments with seeded delays, delivers arrivals, lists in-transit and delayed shipments
`Supplier`	`shipments.py`	Dataclass	Supplier ID, name, lead time and reliability
`Shipment`	`shipments.py`	Dataclass	One replenishment shipment with its

Class	Module	Kind	Responsibility
			expected, arrival and actual days; computes `delay_days` and `days_late(day)`
`ShipmentStatus`	`shipments.py`	Enum (`str`)	`IN_TRANSIT`, `DELIVERED`
`analytics`	`analytics.py`	Module of functions	`find_delayed_shipments`, `find_stockouts`, `find_low_stock`, `find_excess`, `order_metrics`, `build_report`, `format_report`

Relationships

Relationship	Type	Multiplicity	Meaning
`Simulation` → `Inventory`, `OrderManager`, `ShipmentManager`	Composition	1 to 1 each	Created in `Simulation.__init__` and live as long as the simulation
`Simulation` → `ScheduledOrder`	Composition	1 to 0..*	Demand queued by day in `_schedule`; removed when its day is simulated
`Simulation` → `DayLog`	Composition	1 to 0..*	One log appended per `step()`
`Simulation` ⇢ `analytics`	Dependency	–	Calls `build_report` and `format_report`
`OrderManager` → `Inventory`, `ShipmentManager` → `Inventory`	Association (shared)	1 to 1	Both hold a reference to the *same* `Inventory` instance
`OrderManager` → `Order`	Composition	1 to 0..*	Orders stored in `_orders` by `order_id`
`ShipmentManager` → `Supplier`, `Shipment`	Composition	1 to 0..*	Stored in `_suppliers` and `_shipments` by ID
`Shipment` ⇢`Supplier`	Dependency by ID	0..* to 1	`Shipment.supplier_id` refers to a registered supplier
`Inventory` → `StockPolicy`	Composition	1 to 0..*	One policy per (warehouse, SKU); defaults are used when none is set
`Order` → `OrderStatus`, `Shipment` → `ShipmentStatus`	Association	–	Current lifecycle state
`DayLog` → `Order`, `Shipment`	Aggregation	0..*	References to objects owned by the managers

Design notes

- **Records versus behaviour.** Plain records (`Order`, `Shipment`, `Supplier`, `StockPolicy`, `ScheduledOrder`, `DayLog`) are dataclasses; behaviour lives in the manager classes. `StockPolicy` and `ScheduledOrder` are frozen, so they cannot be changed after creation.
- **Validation in constructors.** `StockPolicy`, `Order`, `Supplier` and `Shipment` validate their fields in `__post_init__` and raise `ValueError` for invalid values (SCM-SR-001).
- **IDs instead of object links between modules.** Orders and shipments refer to warehouses, SKUs, customers and suppliers by string IDs (e.g. `WH-NORTH`, `WIDGET`, `C-ACME`, `SUP-RELIABLE`). This keeps the modules independent and the data easy to print and compare.
- **Enums that are also strings.** `OrderStatus` and `ShipmentStatus` subclass `str`, so Analytics can count statuses whether it receives enum members or plain strings.

3.4 Component Descriptions

Each component below is described by its responsibility, the interface it provides and the interfaces it requires, its internal data, its key design decisions, and the requirements and tests it traces to. Detailed call-by-call API definitions are in §4.3.

3.4.1 Inventory, Simulation Engine, CLI and CI Pipeline

Inventory

Item	Description
File / owner	`src/scm_sim/inventory.py` / Rohith Karning
Layer	Domain
Responsibility	Single source of truth for stock: quantity per (warehouse, SKU), total capacity per warehouse, and a stock policy per item. Decides whether an item needs a reorder or holds excess stock.
Provided interface	`Inventory`: `level`, `add`, `remove`, `is_stockout`, `register_warehouse`, `total_in_warehouse`, `set_policy`, `get_policy`, `needs_reorder`, `is_excess`, `reorder_suggestions`, `snapshot`. `StockPolicy` (frozen dataclass). Exceptions `InsufficientStockError`, `CapacityExceededError`.
Required interface	None. Inventory imports no other `scm_sim` module.
Used by	Order Manager (`level`, `remove`), Shipment Manager (`add`), Simulation Engine (set-up, `reorder_suggestions`, `total_in_warehouse`, `snapshot`)
Requirements	SCM-F-001, SCM-F-002, SCM-F-003, SCM-F-004; supports SCM-SR-001, SCM-SR-002, SCM-SR-007
Tests	`tests/test_inventory.py` (20 test functions, 23 test runs including parametrised cases); TC-INV-01, TC-INV-02

Internal data

Attribute	Type	Content
`_stock`	`dict[(warehouse, sku), int]`	Current quantity of each item. A key is created the first time the item is stocked and kept even when its quantity falls to 0, so a

Attribute	Type	Content
		stockout stays visible in snapshots.
`_capacity`	`dict[warehouse, int]`	Capacity of each registered warehouse. A warehouse with no entry has unlimited capacity.
`_policies`	`dict[(warehouse, sku), StockPolicy]`	Explicit policies. `get_policy` returns `StockPolicy()` (20 / 100 / 500) for items without one.

Key design decisions

1. **Check before change.** `add` validates the quantity and the capacity, and `remove` validates the quantity and the available stock, *before* updating `_stock`. A rejected call therefore leaves stock exactly as it was (SCM-SR-002).
2. **Capacity is total units per warehouse.** `add` compares `total_in_warehouse(warehouse) + qty` with the capacity, so all SKUs share one limit. Adding exactly up to capacity is allowed.
3. **Immutable policies.** `StockPolicy` is a frozen dataclass that validates itself in `__post_init__` (no negative values; `max_level ≥ reorder_point`). A policy is changed by replacing it with `set_policy`, never by editing it.
4. **Deterministic suggestions.** `reorder_suggestions` covers every *tracked* item (stocked at least once, or with a policy) and is sorted by warehouse and then SKU. This makes replenishment order, and therefore shipment IDs, reproducible (SCM-NF-002).
5. **Copies out, never references.** `snapshot()` returns `dict(self._stock)`, a new dictionary, so readers such as Analytics cannot change stock (SCM-SR-007).

Constraints and limitations. Capacity is enforced only when stock is added; registering a capacity below the stock already held is accepted. Warehouses and SKUs are not registered centrally, so an unknown warehouse simply has 0 stock and no capacity limit at this level. The Simulation Engine adds the warehouse-existence check (SCM-SR-004).

Simulation Engine

Item	Description
File / owner	`src/scm_sim/simulator.py` / Rohith Karning
Layer	Orchestration
Responsibility	Holds one scenario and advances it one day at a time. Wires Inventory, Order Manager, Shipment Manager and Analytics together, controls the order of events inside a day, replenishes stock automatically and keeps the history of daily reports. Provides the ready-made demo scenario.
Provided interface	`Simulation`: `add_supplier`, `add_warehouse`, `set_policy`, `add_stock`, `schedule_order`, `step`, `run`, `final_summary`, `all_orders`; module-level functions `build_demo(seed)`, `describe_day(log, report)` and `metrics_rows(history)`. Dataclasses `ScheduledOrder` and `DayLog`.
Required interface	Inventory, Order Manager, Shipment Manager (public methods only), Analytics `build_report`, `format_report`, `summarize_run` and `format_run_summary`, standard library `random.Random`
Used by	CLI; tests in `tests/test_simulator.py`

Item	Description
Requirements	SCM-F-040, SCM-F-041, SCM-F-042, SCM-F-044; supports SCM-NF-002, SCM-SR-001, SCM-SR-004
Tests	`tests/test_simulator.py` (10 test functions); TC-SIM-01, TC-SIM-02, TC-REP-01

Internal data

Attribute	Type	Content
`day`	`int`	Last simulated day; 0 before the first step
`inventory`, `orders`, `shipments`	`Inventory`, `OrderManager`, `ShipmentManager`	Created in the constructor. Both managers share the same `Inventory`; the Shipment Manager receives `random.Random(seed)`.
`_warehouses`	`dict[warehouse, capacity]`	Registered warehouses (used for validation and capacity trimming)
`_suppliers`	`dict[supplier_id, Supplier]`	Registered suppliers (used to validate `set_policy`)
`_item_supplier`	`dict[(warehouse, sku), supplier_id]`	The one supplier that replenishes each item
`_schedule`	`dict[day, list[ScheduledOrder]]`	Future customer demand; a day's entry is removed when that day is simulated
`history`	`list[dict]`	One Analytics report per simulated day, in order
`logs`	`list[DayLog]`	One event record per simulated day, in order

The daily step. `step()` increments `day`, then runs five stages in a fixed order:

Stage	Call(s)	Why at this point
1. Arrivals	`shipments.process_arrivals(day)`	Goods that arrive today can be used today.
2. Back-order retry	`orders.process_open(day)`	Older open orders are served before today's new demand, so earlier customers are not starved by later ones.
3. Today's orders	`orders.create(…)` then `orders.try_fulfil(order, day)` for each scheduled order	New demand is served from the stock remaining after back-order retries.
4. Replenishment	`_replenish(day)` using `inventory.reorder_suggestions()`, `shipments.in_transit()`, `shipments.dispatch(…)`	Reorder decisions see the end-of-day stock after all demand.
5. Report	`build_report(inventory.snapshot(), shipments.delayed(day), all_orders(), day, …)`	The report shows the final state of the day.

Replenishment algorithm (`_replenish`). For each `(warehouse, sku, reorder_quantity)` from `reorder_suggestions()` (sorted): skip the item if it has no assigned supplier or already has a shipment in transit; otherwise compute `free = capacity − total_in_warehouse(warehouse) − units in transit to that warehouse − units dispatched earlier in this step`, set `quantity = min(reorder_quantity, free)`, and dispatch only if `quantity > 0`. Stock only increases through arrivals, so a shipment that fitted when dispatched still fits when it arrives, and `CapacityExceededError` cannot be raised during `process_arrivals`.

Key design decisions

1. **One orchestrator.** All cross-module calls during a run happen in `step()`; no domain component calls another except through the shared Inventory. Event order is therefore defined and testable in one place.
2. **Validate at the boundary.** `schedule_order` rejects past days, quantities ≤ 0 and unknown warehouses before queuing anything; `set_policy` and `add_stock` reject unknown warehouses and suppliers with `KeyError`. Invalid demand never reaches the Order Manager.
3. **Public interfaces only.** The engine gets every order via `by_status` over all `OrderStatus` values (sorted by ID), and late shipments via `delayed(day)`. It never reads another module's private attributes.
4. **Reproducibility.** The only randomness is the seeded generator passed to the Shipment Manager; all iteration orders are sorted or insertion-ordered. Same seed and scenario give identical `history` (SCM-NF-002).
5. **Global report thresholds.** The constructor's `reorder_point` and `max_level` (defaults 20 and 500) are passed to `build_report`, because Analytics accepts one threshold for all items. Replenishment itself uses the per-item policies.

Demo scenario (`build_demo(seed=42)`). Suppliers `SUP-RELIABLE` (lead time 2, reliability 1.0) and `SUP-FLAKY` (lead time 2, reliability 0.3); warehouses `WH-NORTH` (capacity 1200) and `WH-SOUTH` (capacity 500); items `WIDGET` and `GIZMO` at `WH-NORTH`, `GADGET` and `WIDGET` at `WH-SOUTH`; 14 orders from customers `C-ACME`, `C-BOLT`, `C-CORE` and `C-DYNA` over days 1–10. It is designed to show every condition Analytics detects: `GADGET` at `WH-SOUTH` sells out on day 2 and a day 3 order is back-ordered; `SUP-FLAKY` shipments arrive late; `GIZMO` starts at 650 against a max level of 500.

CLI

Item	Description
File / owner	`src/scm_sim/__main__.py` / Rohith Karning
Layer	Presentation
Responsibility	Command-line entry point: parses options, runs the demo scenario, prints the day-by-day log and final report, and optionally prints a run summary or exports daily metrics as CSV
Provided interface	`scm-sim` or `python -m scm_sim [--days N] [--seed S] [--summary] [--csv PATH]` (defaults 10 and 42); function `main(argv=None) -> int`
Required interface	Simulation Engine `build_demo`, `Simulation.run`, `Simulation.logs`, `Simulation.final_summary`, `metrics_rows(history)`, `describe_day`; Analytics run-summary functions; standard library `argparse`, `csv`
Requirements	SCM-F-043, SCM-F-044; supports SCM-F-034 and SCM-NF-005
Tests	`test_cli_main_runs`,

Item	Description
	`test_cli_runs_as_module` in `tests/test_simulator.py`; TC-SIM-02

Behaviour. `main` parses the arguments with `argparse`; a `--days` value below 1 is reported with `parser.error`, which prints the usage and exits with code 2, and non-integer values are rejected by `argparse` in the same way. It then prints a header line, one block per day produced by `describe_day` (arrivals, fulfilled orders, back-orders, back-orders filled, reorders, and a `flags:` summary line), a blank line and the formatted final report, and returns 0. `main` takes an optional `argv` list so tests can call it directly without starting a new process.

Design decisions. The CLI contains no business logic; it only connects the user to the Simulation Engine, so all behaviour it shows is already covered by the engine's tests. Output is plain text, so it can be compared between runs to check reproducibility (TC-REP-01).

CI Pipeline

Item	Description
File / owner	`.github/workflows/ci.yml` / Rohith Karning
Layer	Development infrastructure (not part of the runtime product)
Responsibility	Run the full automated test suite with coverage on every change proposed to, or merged into, `main`, and provide the required status check that gates merging
Triggers	`push` to `main`; `pull_request` targeting `main`
Job	`test` on `ubuntu-latest`: `actions/checkout@v5` → `actions/setup-python@v6` (Python 3.11) → install test requirements and editable package → `pytest -v --cov=scm_sim --cov-branch --cov-fail-under=90` with terminal, HTML, XML and JUnit reports
Requirements	SCM-NF-003, SCM-SR-005; supports SCM-NF-004 (Linux run) and SCM-SO-2
Evidence	20 of 20 workflow runs successful up to run #20 on commit `b3702ef` (11-10-2026): 73 tests passed, 97% line coverage

Design decisions. The job name `test` is configured as a required status check in the branch protection rule for `main`, so a pull request cannot be merged until the suite passes on its latest commit. Running on Linux in CI while developers use Windows gives a cross-platform check on every change. Coverage is measured and printed on every run; the 90% target of SCM-NF-003 is checked by reading that report (TC-COV-01), because the workflow does not set a `--cov-fail-under` limit.

3.4.2 Order Manager (`scm_sim.orders`)

Responsibility

The `OrderManager` component is the domain authority for customer demand lifecycle management. It encapsulates the creation, indexing, state tracking, fulfillment, retry scheduling, and cancellation of customer orders. It serves as an intermediary between simulation demand events and warehouse physical stock, enforcing the business rule that order quantities must be fulfilled in their entirety or backordered without partial deduction. When invoked during the daily simulation cycle, open back-orders are retried first (oldest-first FIFO) before new daily demand is admitted, ensuring fairness to earlier customers (#69). Furthermore, order creation validates demand quantities prior to identifier allocation so invalid requests fail fast without creating gaps in the sequential identifier counter (#68).

Inputs and Outputs

- **Inputs:**
  - Order creation parameters: `customer_id` (str), `warehouse_id` (str), `sku` (str), `quantity` (int > 0), and simulation `day` (int).
  - Fulfillment and retry invocations: simulation `day` (int) and target `Order` instances.
  - Query and cancellation parameters: `order_id` (str) and `OrderStatus` filter criteria.
- **Outputs:**
  - Newly instantiated `Order` records assigned sequential IDs (ORD-`0001`, ORD-`0002`, …).
  - Boolean fulfillment status flags (`True` for success, `False` for shortage/ineligibility).
  - Lists of newly fulfilled `Order` objects returned by batch retry operations (`process_open()`).
  - Filtered collections of orders partitioned by current lifecycle state (`by_status()`).

Dependencies

- **Depends on:** `scm_sim.inventory.Inventory`. The `OrderManager` directly queries `Inventory.level(warehouse, sku)` to evaluate stock availability and calls `Inventory.remove(warehouse, sku, qty)` to atomically deduct stock upon successful fulfillment.
- **Depended on by:** `scm_sim.simulator.Simulation`. The simulation orchestrator calls `OrderManager.create()`, `OrderManager.try_fulfil()`, and `OrderManager.process_open()` during its daily execution cycle.
- **Read by:** `scm_sim.analytics.order_metrics()`. The analytics engine consumes lists of `Order` entities to evaluate fill rate performance.

Requirements Satisfied

- SCM-F-`010`: Sequential order ID assignment (ORD-`0001`…) with pre-allocation positive quantity validation preventing sequence gaps (#68).
- SCM-F-`011`: Full-quantity fulfillment against inventory; automatic transition to BACKORDERED upon stock shortfall.
- SCM-F-`012`: FIFO batch retry of open (PENDING and BACKORDERED) orders sorted by `(created_day, order_id)`, evaluated prior to admitting new same-day customer demand (#69).
- SCM-F-`013`: Guarded cancellation (forbidding cancellation of FULFILLED orders), unique lookup (`get`), and status filtering (`by_status`).
- SCM-SR-`001`, SCM-SR-`003`: Fail-fast input validation and protected state transitions.

3.4.3 Shipment Manager Component

Component Overview

The **Shipment Manager** (`scm_sim.shipments.ShipmentManager`) is the domain component responsible for supplier registries, dispatching inbound replenishment shipments, simulating lead times and stochastic transit delays, delivering arriving shipments into warehouse inventory, and tracking overdue physical movements.

Architectural Responsibilities

1. **Supplier Registration and Validation:** Maintains the registry of external supply partners, enforcing uniqueness and validating operational parameters (non-negative lead times and valid reliability probability bounds).
2. **Replenishment Dispatching:** Instantiates and schedules inbound shipments from suppliers to destination warehouses, generating sequential IDs (SHP-`0001`, SHP-`0002`, …), calculating scheduled arrival days, and initializing status to `IN_TRANSIT`.
3. **Stochastic Transit Delay Simulation:** Employs an injected pseudorandom number generator (`random.Random`) to simulate realistic supply chain delivery variability based on individual supplier reliability ratings.

4. **Physical Inbound Receiving (Arrival Processing):** Delivers due shipments on or after their arrival day, transferring stock into warehouse inventory solely via `Inventory.add()` and transitioning status to `DELIVERED`.
5. **Overdue and In-Transit Inquiries:** Exposes query operations for active in-transit stock movements and overdue shipments for downstream analytics and exception monitoring.

Inputs and Outputs

- **Inputs:**
  - `Supplier` definitions: `supplier_id` (str), `name` (str), `lead_time_days` (int >= 0), `reliability` (float in [0.0, 1.0]).
  - Dispatch parameters: `supplier_id`, `warehouse_id`, `sku`, `quantity` (> 0), `day` (current simulation day), and optional explicit `delay` (int).
  - Daily progression triggers: `day` (current simulation day).
- **Outputs:**
  - Created `Shipment` instances.
  - Lists of `Shipment` objects delivered on a given day (`process_arrivals(day)`).
  - Lists of currently in-transit shipments (`in_transit()`).
  - Lists of currently overdue shipments (`delayed(current_day)`).

Collaborators and Dependencies

- **`scm_sim.inventory.Inventory`:** Collaborates with inventory by invoking `inventory.add(warehouse_id, sku, quantity)` when inbound shipments reach their destination. Never directly manipulates internal inventory dictionaries.
- **`random.Random` (Python Standard Library):** Utilizes an injected instance of `random.Random` (defaulting to seed 42) to ensure repeatable, deterministic delay modeling across simulation executions.

Internal State Structures

- `_suppliers: Dict[str, Supplier]`: Hash map storing registered `Supplier` objects keyed by `supplier_id`.
- `_shipments: Dict[str, Shipment]`: Hash map storing all created shipments keyed by `shipment_id`.
- `_next_id: int`: Internal auto-increment integer counter ensuring sequential, non-colliding shipment IDs formatted as `SHP-{self._next_id:04d}`.

Delay Model Specification

The delivery timeline for each dispatched shipment is computed as follows: 1. **Expected Arrival Day:** `expected_day = day + supplier.lead_time_days` 2. **Delay Calculation:** - If an explicit `delay` integer is provided: `delay = delay` - If `delay is None`:

$$\text{delay} = \begin{cases} \text{rng.randint}(1,3), & \text{if rng.random}() > \text{supplier.reliability} \\ 0, & \text{otherwise} \end{cases}$$

3. **Actual Arrival Day:** `arrival_day = expected_day + delay` 4. **Reliability Invariant:** When `supplier.reliability == 1.0`, `rng.random() > 1.0` is mathematically impossible, guaranteeing `delay = 0` and on-time arrival.

Traceability to Requirements

- **Functional Requirements:** SCM-F-020, SCM-F-021, SCM-F-022, SCM-F-023.
- **Security & Quality Requirements:** SCM-SR-001 (fail-fast input validation), SCM-SR-004 (KeyError on unknown supplier), SCM-NF-002 (deterministic reproducibility via seeded PRNG).

3.4.4 Analytics Component

Component Overview

The **Analytics** component (`scm_sim.analytics`) is the read-only observation layer of the simulator. It transforms the plain state snapshots produced by the domain modules into the operational findings that the Supply Chain Analyst needs: which inbound shipments are late and by how many days, which items are stocked out, low or in excess, how the order book is performing, and how metrics aggregate across a complete run. It is implemented as nine module-level functions, all of which are **pure**: they read their arguments, return new data structures, and never mutate any argument or any module-level variable. The component holds **no internal state** and is instantiated never — it is used only as a namespace of functions.

Architectural Responsibilities

1. **Late-shipment detection (`find_delayed_shipments`):** Compares each shipment's expected arrival day against its actual arrival day when it has been delivered, or against the current simulation day when it is still in transit, and reports only those that are strictly late, ordered worst first.
2. **Inventory health classification (`find_stockouts`, `find_low_stock`, `find_excess`):** Partitions the stock snapshot into three disjoint exception classes using two configurable thresholds — stockouts at exactly zero, low stock in the open interval up to and including the reorder point, and excess strictly above the max level with the surplus quantity reported alongside.
3. **Order performance aggregation (`order_metrics`):** Counts orders per lifecycle status and derives the fill rate as fulfilled orders over eligible orders, rounded to two decimal places.
4. **Report composition (`build_report`):** Combines the outputs of the five classifiers above into a single daily report dictionary with a fixed key set, so that the daily time-step has exactly one artefact to store and print.
5. **Report presentation (`format_report`):** Renders a report dictionary as a stable multi-line text block with one counted section per key and an explicit "No issues" marker for empty sections, so that an absence of problems is visible rather than ambiguous.
6. **Run summary aggregation (`summarize_run`):** Aggregates day counts, issue-day counts, peak delayed shipments and the final fill rate from retained daily reports (SCM-F-034).
7. **Run summary presentation (`format_run_summary`):** Renders the aggregate as a readable multi-line summary for the CLI.

Public Interface

The component exports exactly nine functions and one type hint convention; it exports no classes, no constants and no exceptions of its own.

Function	Returns
`find_delayed_shipments(shipments, current_day)`	`list[dict]` with keys `shipment_id`, `sku`, `warehouse_id`, `days_late`
`find_stockouts(stock)`	`list[tuple[str, str]]` of `(warehouse_id, sku)`
`find_low_stock(stock, reorder_point=20)`	`list[tuple[str, str, int]]` of `(warehouse_id, sku, qty)`
`find_excess(stock, max_level=500)`	`list[tuple[str, str, int, int]]` of `(warehouse_id, sku, qty, surplus)`
`order_metrics(orders)`	`dict` with keys `total`, `pending`, `fulfilled`, `backordered`, `cancelled`, `fill_rate`
`build_report(stock, shipments, orders, current_day, reorder_point=20, max_level=500)`	`dict` with keys `day`, `delayed_shipments`, `stockouts`, `low_stock`, `excess`, `orders`
`format_report(report)`	`str` — the rendered multi-line report
`summarize_run(history)`	`dict` with run length, issue-day counts, peak delayed-

Function	Returns
	shipment count and final fill rate
`format_run_summary(summary)`	`str` — the rendered multi-line run summary

The complete signature, parameter, return and example detail for each call is given in SAD §4.3.4.

Inputs and Outputs

- **Inputs:**
  - Stock: the `dict[{(warehouse_id, sku): qty}]` mapping returned by `Inventory.snapshot()` — an independent copy, so Analytics never observes live inventory state.
  - Shipments: a `list` of `Shipment` objects exposing `shipment_id`, `sku`, `warehouse_id`, `expected_day` and `actual_day`. Only these five attributes are read; the Shipment Manager's internal collections are never reached.
  - Orders: a `list` of `Order` objects exposing `status` only. The status may be an `OrderStatus` enumeration member or the equivalent plain string, and both forms may be mixed in one list.
  - Day: the current integer simulation day.
  - Thresholds: `reorder_point` and `max_level`, defaulted to 20 and 500.
- **Outputs:**
  - New lists, dictionaries and strings. Every returned container is freshly allocated, so the caller may store, mutate or discard the result without affecting the simulator.

Collaborators and Dependencies

The Analytics component has **no collaborators and no dependencies**. It imports only `from __future__ import annotations` and `from typing import Any` from the Python standard library. It does not import `Inventory`, `OrderManager`, `ShipmentManager` or `Simulation`, and those modules do not import it; the coupling is one-way and expressed entirely through the `Simulation` orchestration layer, which calls the daily-report functions during each time-step and the run-summary functions when requested by the CLI. This satisfies the dependency direction defined in SAD §3.5 (CLI → Simulation → domain modules; analytics depends on nothing) and is what makes the component trivially unit-testable: the test suite constructs stand-ins with `types.SimpleNamespace` rather than building a simulation.

Internal State Structures

None. There are no instance attributes, no module-level mutable globals and no caches. Every function is referentially transparent with respect to its arguments, which is why the component contributes no state that could be corrupted and raises no security requirement of its own beyond SCM-SR-007 (returned data is a copy).

Key Algorithms

1. **Days-late computation.** For each shipment the component selects `end_day = actual_day if actual_day is not None else current_day`, then `days_late = end_day − expected_day`, and retains only entries where `days_late > 0`. Selecting on `is not None` rather than truthiness matters: a shipment delivered on day 0 records `actual_day = 0`, which must be treated as delivered rather than as in transit.
2. **Ordering.** The delayed list is sorted on the key `(-days_late, shipment_id)`. The negated `days_late` gives descending lateness, and `shipment_id` is included as a deterministic tie-break so that two shipments that are equally late always appear in the same order between runs — required by SCM-NF-002. The three stock classifiers use Python's `sorted` on tuple literals, which sorts first by `warehouse_id` and then by `sku`, giving a stable, human-readable ordering for free.
3. **Threshold comparison.** The low-stock test is the strict-and-inclusive pair `0 < qty <= reorder_point` and the excess test is the strict `qty > max_level`. Together with the exact test `qty == 0` for stockouts, the three classes are disjoint and together cover every non-negative quantity, so no item is ever reported twice or omitted.

4. **Status normalisation.** Each order status is read as `getattr(o.status, "value", o.status)`, which unwraps an enumeration member to its string value and passes a plain string through unchanged. The resulting value is then looked up in the fixed counts dictionary, so a status outside the four known values is ignored rather than raising — Analytics never fails on data it does not recognise.
5. **Fill rate.** `fill_rate = round(fulfilled / (total - cancelled), 2)` when `total - cancelled > 0`, and `0.0` otherwise. Cancelling an order removes the demand rather than counting it as a service failure, so cancelled orders are excluded from the denominator; the guarded division prevents `ZeroDivisionError` both for an empty order book and for an order book in which every order has been cancelled.

Design Decisions and Alternatives Considered

Decision	Rationale	Alternative rejected
Pure functions in one module, rather than an `Analytics` class with methods	Analytics needs no identity, no configuration and no lifetime. Pure functions can be imported and called directly, are trivially testable in isolation, and cannot leak state between the many daily reports the Simulation retains.	An `Analytics` class instance mirroring the other components would have added an empty constructor and an object whose only state would be unused; it would also have forced the Simulation to own an instance it never reads from.
Thresholds supplied as function parameters rather than read from `StockPolicy`	This keeps the component dependency-free and makes the thresholds explicit and visible at every call site. It also allows a report to be computed with different thresholds from the ones in force, which is what the Analytics tests exercise.	Reading the per-item `StockPolicy` held by `Inventory` would have coupled Analytics to the Inventory component and to its data structures, and would have changed the signature and semantics of three of the seven functions. The consequence of this decision — one global threshold for all items, where Inventory holds per-item policies — is recorded as a known limitation in SAD §4.6.
Tuples rather than small objects for stock findings	`(warehouse_id, sku)` and its extensions are hashable, sortable, cheap to compare in tests and print directly. No attribute access or constructor is needed.	A `StockAlert` dataclass would be more self-describing but would add a class, a constructor and no behaviour; the reporting code would still destructure it field by field.
Reports returned as `dict` and rendered separately	Keeping `build_report` and `format_report` apart means the data and the presentation can be verified and consumed independently: the Simulation stores the dictionaries, the CLI and tests render text.	Rendering inside `build_report` would destroy the structured data, preventing the Simulation from retaining comparable day-by-day reports and making the reproducibility test in TC-REP-01 impossible.
Graceful handling of unknown order statuses	Ignoring an unrecognised status keeps `order_metrics` total and prevents a malformed order from aborting the whole daily report, which the Simulator would surface as a hard failure at day *n*.	Raising on an unknown status would be stricter, but Analytics is read-only and has no way to repair the data; failing the entire run over a single unexpected value is a worse outcome for an observational component.

Traceability to Requirements

- **Functional Requirements:** SCM-F-030 (`find_delayed_shipments`), SCM-F-031 (`find_stockouts`, `find_low_stock`, `find_excess`), SCM-F-032 (`order_metrics`), SCM-F-033 (`build_report`, `format_report`).
- **Non-Functional Requirements:** SCM-NF-002 (deterministic ordering makes same-seed runs identical), SCM-NF-005 (`format_report` prints a count per section and "No issues" for empty sections), SCM-NF-003 (contributes to the measured coverage of the `scm_sim` package).
- **Security Requirements:** SCM-SR-007 only, and satisfied by construction — every returned list, tuple and dictionary is newly allocated, so a caller that mutates a report cannot change any internal state. The component performs no input validation and raises no exceptions because it mutates nothing: every operation it offers is safe to apply to any argument it can be given, so SCM-SR-001 has nothing to enforce here.

Known Limitations

1. **A single global threshold pair.** `find_low_stock` and `find_excess` take one `reorder_point` and one `max_level` for the whole snapshot, whereas Inventory holds a `StockPolicy` per (warehouse, SKU) item. A scenario whose items have different policies will be reported against the global defaults unless the caller overrides them.
2. **No history or trend.** Each call analyses a single point in time. The component reports the state on one day and holds no notion of a previous day, so it cannot itself compute a trend, an average delay or a time-to-resolution; any such figure would have to be derived by a caller comparing retained reports.
3. **Tuple-heavy output.** The stock classifiers return positional tuples, which are compact and cheap but not self-describing at the call site; a caller must remember the field order of each return type.
4. **No persistence or export.** Reports exist only as in-memory dictionaries and strings; they are printed by the CLI and retained by the Simulation but cannot be written to a file or read back later.
5. **Edge case not special-cased.** A shipment with `actual_day` recorded as day 0 is correctly treated as delivered because the code tests `is not None`, but this is the only such guard and a reader must know why; there is no general notion of "sentinel" values in the interface.

3.5 Architecture Pattern and Rationale

3.5.1 Chosen Pattern: Layered, Modular Architecture in One Package

The simulator uses a **layered, modular architecture** inside a single Python package (`scm_sim`). Each layer has one kind of responsibility, and each module inside a layer owns its own state and rules behind a small public interface.

Layer	Component(s)	Responsibility	May depend on
Presentation	CLI (`__main__.py`)	Parse command-line options; print the day-by-day log and final report; optionally print a run summary or export daily metrics	Orchestration
Orchestration	Simulation Engine (`simulator.py`)	Hold the scenario; run the discrete daily time step; call the domain modules in a fixed order; keep the report history	Domain, Analytics
Domain	Inventory (`inventory.py`), Order Manager (`orders.py`), Shipment Manager (`shipments.py`)	Own supply-chain state (stock, orders, shipments) and enforce its rules	Inventory only

Layer	Component(s)	Responsibility	May depend on
Analytics	Analytics (`analytics.py`)	Pure functions that turn a stock snapshot, shipments and orders into a report; never change state	Nothing

Dependency rule. Dependencies point one way: **CLI → Simulation Engine → domain modules and Analytics**. Within the domain layer, the Order Manager and the Shipment Manager depend only on Inventory, and never on each other. Analytics depends on nothing in the package; it reads plain data handed to it. There are no cyclic imports, and the source code confirms this (`orders.py` and `shipments.py` import only `scm_sim.inventory`; `analytics.py` imports no `scm_sim` module).

Time model. Inside the orchestration layer the system is a **discrete time-step simulation**: time advances in whole days, and every day runs the same five stages in the same order (arrivals → open back-order retry → today's orders → replenishment → report; see §3.4.1 and §4.1).

Shared state. The only state shared between domain modules is the single `Inventory` object, passed to the `OrderManager` and `ShipmentManager` constructors. The Simulation Engine decides *when* each manager reads or changes it.

3.5.2 Rationale

Driver	How the pattern addresses it
Parallel team development (four people, one module each)	Each domain and analytics module was specified with a fixed interface, built and unit-tested on its own feature branch, and merged independently (PRs #2, #3, #5, #6). The Simulation Engine was integrated last (PR #7) without changing any other module.
Testability (SCM-NF-003)	Lower layers do not depend on higher ones, so every module is tested in isolation: Inventory with no other module, the managers with a real `Inventory`, Analytics with simple stand-in objects. Integration and CLI behavior are covered by simulator and packaging tests. Final evidence: 100 tests passed; 99.79% line, 99.17% branch and 99.66% combined coverage.
Correct event ordering (SCM-F-041)	Only `Simulation.step()` calls the domain modules during a run, so the order of events within a day is defined, documented and tested in one place.
Determinism and reproducibility (SCM-NF-002)	Randomness is confined to one seeded `random.Random` in the Shipment Manager, and every loop iterates in a sorted or insertion order. A discrete time step has no timing or threading effects.
Data integrity (SCM-SO-1, SCM-SR-001, SCM-SR-002)	Each domain module validates input and protects its own invariants, and Analytics is read-only and receives a copy of stock. A bug in reporting cannot corrupt simulation state.
Simplicity and portability (SCM-NF-004)	An installable package with no runtime third-party dependencies and no services to deploy. Windows and Linux are tested; macOS portability remains unverified.
Understandability for evaluation	The layers correspond one-to-one with the problem statement: manage inventory, orders and shipments (domain), simulate movement over time (orchestration), identify delays, stockouts and excess

Driver	How the pattern addresses it
	(analytics).

3.5.3 Alternatives Considered and Rejected

Alternative	Advantages	Reasons for rejection
(a) Event-driven / message-queue simulation (e.g. a discrete-event library such as SimPy, with events scheduled at arbitrary times)	More realistic timing (events within a day, continuous lead times); scales to many concurrent processes	Adds a third-party runtime dependency, which violates the standard-library-only constraint. Ordering of simultaneous events becomes implicit, which makes tests harder to write and read and makes deterministic results harder to guarantee. Daily granularity is sufficient for the problem statement.
(b) Web application with a database (REST back end, persistent storage, browser front end)	Multi-user access, persistent scenarios, graphical dashboards	Out of scope for a simulator. Adds deployment, configuration and a large security surface (authentication, injection, network exposure) with no benefit to the problem statement. Much slower to test, and data would no longer be reproducible from a seed alone.

3.5.4 Consequences and Trade-offs

Positive

- Modules are small and independently testable; each team member could own and defend one module.
- Behaviour is fully reproducible from the seed and the scenario.
- The package is installable with `pip install .` and has no runtime third-party dependencies.

Negative / accepted trade-offs

- **Coarse time.** Events within a simulated day use the fixed order stated above rather than specific times; this is the selected daily simulation model.
- **In-memory simulation state.** Scenarios are built in Python code and simulation state is not persisted; users may export daily metrics to CSV.

3.6 Technology Stack and Data Stores

3.6.1 Technology Stack

The simulator is engineered intentionally with minimal runtime complexity to maximize execution speed, maintainability, and cross-platform portability.

Technology Dimension	Selection	Version / Standard	Architectural Role & Rationale
Programming Language	Python	≥ 3.11	Core language runtime; uses type hints, dataclasses, enums and standard-library CLI, CSV and random-number facilities.
Packaging	setuptools / `pyproject.toml`	`setuptools>=68` build backend	Installable `src`-layout package `scm-sim`;

Technology Dimension	Selection	Version / Standard	Architectural Role & Rationale
			exposes the `scm-sim` console script and `python -m scm_sim` module entry point. Build tooling is not a runtime dependency.
Runtime Dependencies	Python Standard Library	Standard Library Only	Zero third-party runtime dependencies. Uses `dataclasses`, `enum`, `random`, `argparse`, `csv` and `typing`.
Testing Framework	pytest	≥ 8.0	Automated test discovery, test parameterization, fixture execution, and assertion introspection.
Code Coverage Tool	pytest-cov	≥ 5.0	CI measures statement and branch coverage and fails the test job below the configured 90% coverage threshold.
Continuous Integration	GitHub Actions	`ubuntu-latest`, Python 3.11	Workflow installs test dependencies and the package, then runs the full suite with branch coverage on pushes to `main` and pull requests targeting `main`; test and coverage artifacts are uploaded.
Version Control	Git / GitHub	Git v2.40+	Distributed source control with protected branch policies, peer-reviewed pull requests, and commit traceability.
Modeling & Diagrams	PlantUML	PlantUML 1.2023+	Text-as-code modeling tool generating UML component, class, and sequence diagrams.

3.6.2 Data Stores

Simulation state is held in **in-memory data structures**; there is no database or runtime network storage. The CLI's optional `--csv PATH` flag writes an explicit user-requested metrics export, and CI writes test/coverage artifacts: 1. **Inventory Stock Map:** Stored within `Inventory._stock` as a Python dictionary keyed by composite tuple `(warehouse_id: str, sku: str)` mapping to current integer quantity (`int >= 0`). 2. **Warehouse Capacity Registry:** Stored within `Inventory._capacity` as a Python dictionary keyed by `warehouse_id: str` mapping to maximum integer capacity (`int > 0`). 3. **Stock Policies Registry:** Stored within `Inventory._policies` as a dictionary keyed by `(warehouse_id: str, sku: str)` mapping to `StockPolicy` dataclass instances. 4. **Order Registry:** Stored within `OrderManager._orders` as a dictionary keyed by `order_id: str` (e.g., `"ORD-0001"`) mapping to `Order` dataclass instances. 5. **Shipment Collection:** Stored within `ShipmentManager._shipments` as a Python list containing tracked `Shipment`

dataclass instances. 6. **Supplier Registry:** Stored within `ShipmentManager._suppliers` as a dictionary keyed by `supplier_id: str` mapping to `Supplier` dataclass instances.

The simulation data stores are volatile and reside within process memory. Each new `Simulation` initializes fresh state; CSV and CI artifacts are outputs, not simulation-state stores.

3.7 Risks and Mitigations

This section records the technical and process risks identified during the design of the Supply Chain Management Simulator, the likelihood and impact of each, the mitigation in place or planned, and the owner accountable for it. Likelihood and impact are rated **Low**, **Medium** or **High** on a three-point scale. Risks are derived from the architecture as built (SAD §3.3, §3.5), from the limitations recorded in SAD §4.6, and from the parallel-development and branch-protection workflow the team followed.

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	**Integration conflict from parallel module development.** Four components were built against a fixed interface and integrated only at the end, so an interface drift between `Inventory`, `OrderManager`, `ShipmentManager` and `Simulation` would surface only at integration time.	Medium	High	Each module was specified and unit-tested against its own interface before integration, so defects surface in the owning module's test file first. `Simulation` is the single integration point and runs the complete daily sequence in a fixed order (SAD §3.5), so there is exactly one place where cross-module behaviour is exercised. Integration behaviour is covered by `tests/test_simulator.py` (TC-SIM-01, TC-SIM-02), which is mandatory on every PR.	Rohith Karning (integration)
R2	**Non-deterministic output.** Unseeded or multiply-seeded randomness would make daily reports	Low	High	One `random.Random(seed)` instance is created in `Simulation` and injected into	Pranav Achar (Analytics) / Rohith Karning (seeding)

#	Risk	Likelihood	Impact	Mitigation	Owner
	irreproducible, breaking SCM-NF-002 and making any failure impossible to re-investigate.			`ShipmentManager`; no module calls the module-level `random` functions. The seed defaults to 42 and is overridable with `--seed`. Ordering inside Analytics is fully deterministic, including the `(-days_late, shipment_id)` tie-break. Enforced by `test_same_seed_gives_identical_reports` (TC-REP-01).	
R3	**Invalid state admitted through unchecked input.** Negative stock, an over-capacity warehouse or an impossible status change would undermine SCM-SO-1 and make every downstream report meaningless.	Low	High	Fail-fast validation before any mutation: `ValueError` for invalid arguments, `InsufficientStockError` and `CapacityExceededError` for the two stock invariants, `KeyError` for unknown supplier, warehouse or order references. The invariant enforced throughout is that **any operation which raises leaves state unchanged**. Covered by TC-SEC-01 and TC-INV-01.	Rohith Karning (Inventory) / Rishi J Mule (Shipments) / Rohanth Ganesha C (Orders)

#	Risk	Likelihood	Impact	Mitigation	Owner
R4	**Analytics thresholds diverge from inventory policy.** Analytics applies one global `reorder_point` / `max_level` pair while Inventory holds a `StockPolicy` per item, so a report can flag an item as low stock that Inventory would not reorder.	High	Medium	Accepted deliberately and documented: passing the thresholds as explicit parameters keeps Analytics free of dependencies and makes its behaviour obvious at every call site (SAD §3.4.4). The divergence is recorded as a known limitation in SAD §4.6 with per-item thresholds named as the planned improvement. Mitigated today because the `Simulation` uses the same 20 / 500 defaults that `StockPolicy` applies.	Pranav Achar (Analytics)
R5	**`process_arrivals` aborts mid-pass on a capacity breach.** An arrival that would exceed warehouse capacity raises `CapacityExceededError` from `Inventory.add` and stops the remaining deliveries for that day, leaving some due shipments undelivered.	Low	Medium	The `Simulation` avoids triggering it by trimming each replenishment so that on-hand plus in-transit stock never exceeds the capacity (SCM-F-042, TC-SIM-01). The residual behaviour is documented in SAD §4.6; the planned improvement is for arrival processing to skip and report	Rohith Karning (Inventory / Simulation)

#	Risk	Likelihood	Impact	Mitigation	Owner
				the offending shipment rather than abort the pass.	
R6	**Back-order starvation.** Today's new orders are served before older back-orders, so a steady stream of small orders can indefinitely postpone an old large one.	Medium	Medium	Acceptable within the stated design, which prioritises same-day service; the ordering rule is deterministic, so behaviour is reproducible and visible in the daily report. Recorded in SAD §4.6 with the planned improvement of retrying back-orders before admitting new demand.	Rohanth Ganesha C (Order Manager)
R7	**Order ID consumed before validation.** `OrderManager.create` allocates the next sequential `ORD-0001` identifier before checking the quantity, so a rejected call still advances the counter and leaves a gap in the sequence.	Medium	Low	The `Simulation` validates quantity and day before delegating to `create`, so the CLI path is unaffected and IDs stay contiguous in normal operation. Purely cosmetic and recorded in SAD §4.6; the planned fix is to validate before allocating.	Rohanth Ganesha C (Order Manager)
R8	**Weak test evidence for process and portability requirements.** SCM-NF-004 and SCM-SR-005 cannot be verified by the pytest suite, because they	High	Medium	The evidence is defined but not yet collected: TC-COV-01 retains a manual run on Windows 11 and Linux, and TC-SEC-02 defines the manual branch-	Pranav Achar (Dev Lead) / Rohith Karning (CI)

#	Risk	Likelihood	Impact	Mitigation	Owner
	concern host operating systems and GitHub branch protection rather than code.			protection and secrets checks. Both are scheduled for execution by 20-10-2026 (Test Plan §7) and are carried as status N in the RTM (SRS §8) so that the gap is visible rather than implied. CI already enforces the `test` check on every push and pull request.	
R9	**Unreviewed change reaches `main`.** A direct push or a self-approved pull request could merge untested or untraceable work.	Low	High	`main` is protected: a pull request is required, at least one approval from a non-author is required, the `test` check must pass, the branch must be up to date, and force pushes and branch deletion are blocked, with the rules applied to administrators as well. Changes reach `main` only through the feature-branch and pull-request flow of SAD §3.9. Verified manually by TC-SEC-02.	Rohith Karning (repository owner)
R10	**Single-owner knowledge concentration.** Each module is owned by one team member, so a module becomes hard	Medium	Medium	Mitigated by the fixed, narrow interface of each component and by the API documentation in SAD §4.3,	Team (all members)

#	Risk	Likelihood	Impact	Mitigation	Owner
	to maintain if that member is unavailable, and a defect in an unfamiliar module is slow to diagnose.			which lets any member call a component without reading its source. The layered architecture in SAD §3.5 confines a change to one module in most cases, so cross-module impact is limited. Residual exposure is accepted for a four-person semester project.	
R11	**Runaway runtime on large inputs.** A very large `--days` value or an oversized quantity could make a run take unbounded time or memory.	Low	Medium	Arguments are validated at the CLI (`--days < 1` is a usage error, exit code 2) and every daily loop is bounded by the number of days requested. `Inventory` rejects non-positive quantities and enforces registered capacities. Performance is baselined by TC-PERF-01: a 365-day run completes in roughly 0.02 s against a 1 s budget (SCM-NF-001).	Rohith Karning (CLI / Simulation)
R12	**Secrets or runtime third-party dependencies enter the repository.** A committed token or a	Low	Medium	The simulator imports only the standard library at runtime (`dataclasses`, `enum`, `random`, `argparse`) and	Rishi J Mule (Security Test Engineer)

#	Risk	Likelihood	Impact	Mitigation	Owner
	runtime library would violate SCM-SR-006 and widen the attack surface.			makes no network calls; `requirements.txt` holds test-only dependencies (`pytest`, `pytest-cov`). The repository is searched for tokens as part of the manual TC-SEC-02 check.	
R13	**Documentation drifts from the implementation**. Requirements, design and tests are written after the code, so a later change can leave a documented behaviour that no longer holds.	Medium	Medium	Every requirement, component and test case in these documents names the concrete function, exception and pytest test that implements it, so drift is visible when the named test fails to cover the documented behaviour. Changes reach `main` only after the `test` check passes, and the Requirements Registry is the single source of truth for all IDs, so any divergence must be reconciled against it rather than invented locally.	Team (all members)

3.7.1 Residual Risk Summary

R1, R2, R3, R5, R6, R7, R9, R11 and R12 are all mitigated with controls that are in place today and are covered by passing automated tests. The remaining exposure is concentrated in three places: **R4** and **R10**, which are accepted consequences of the chosen architecture and team size rather than defects; **R13**, which is mitigated by process rather than eliminated; and **R8**, the only risk whose mitigation is scheduled but not yet complete, since the six requirements carried at status N in SRS §8 — SCM-NF-001, SCM-NF-002, SCM-NF-003, SCM-NF-004, SCM-

SR-005 and SCM-SR-006 — await manual execution on 20-10-2026. No risk is assessed as High likelihood with unmitigated High impact.

3.8 Traceability to Requirements

This section maps every requirement defined in the Software Requirements Specification to the architectural component that implements it, using the component names defined in SAD §3.3 and listed in SAD §3.4. It is the design-side counterpart of the Requirements Traceability Matrix in SRS §8: SRS §8 traces every requirement to its test case, and this section traces every requirement to the code that satisfies it. Together they close the loop from requirement to design to verification, and a requirement that appears in neither table is treated as unimplemented.

3.8.1 Functional Requirements

Req ID	Requirement short	Primary component	Supporting component(s)	Design section	Test case
SCM-F-001	Stock per (warehouse, SKU), validated add and remove	**Inventory** (`scm_sim.inventory`)	–	SAD §3.4.1, §4.3.1	TC-INV-01
SCM-F-002	Warehouse capacity with `CapacityExceededError`	**Inventory** (`scm_sim.inventory`)	Simulation Engine (registers capacities)	SAD §3.4.1, §4.3.1	TC-INV-01
SCM-F-003	Per-item stock policy, `needs_reorder`, `is_excess`	**Inventory** (`StockPolicy`)	Analytics (reuses the same default thresholds)	SAD §3.4.1, §4.3.1	TC-INV-02
SCM-F-004	Sorted reorder suggestions and stock snapshot	**Inventory** (`scm_sim.inventory`)	Analytics (consumes the snapshot)	SAD §3.4.1, §4.3.1	TC-INV-02
SCM-F-010	Create orders with sequential `ORD-0001`… IDs	**Order Manager** (`scm_sim.orders`)	–	SAD §3.4.2, §4.3.2	TC-ORD-01
SCM-F-011	Fulfil in full or back-order, no partial fulfilment	**Order Manager** (`scm_sim.orders`)	Inventory (stock check and deduction)	SAD §3.4.2, §4.3.2	TC-ORD-01
SCM-F-012	Retry open orders oldest-first	**Order Manager** (`scm_sim.orders`)	Simulation Engine (invokes the retry pass)	SAD §3.4.2, §4.3.2	TC-ORD-02
SCM-F-013	Cancel, look up by ID, list by status	**Order Manager** (`scm_sim.orders`)	–	SAD §3.4.2, §4.3.2	TC-ORD-02
SCM-F-020	Register suppliers with lead time and reliability	**Shipment Manager** (`scm_sim.shipments`)	–	SAD §3.4.3, §4.3.3	TC-SHP-01
SCM-F-021	Dispatch shipments with sequential `SHP-0001`… IDs	**Shipment Manager** (`scm_sim.shipments`)	–	SAD §3.4.3, §4.3.3	TC-SHP-01
SCM-F-022	Model	**Shipment**	Simulation	SAD §3.4.3,	TC-SHP-02

Req ID	Requirement short	Primary component	Supporting component(s)	Design section	Test case
	stochastic delays from supplier reliability	**Manager** (`scm_sim.shipments`)	Engine (supplies the seeded `random.Random`)	§4.3.3	
SCM-F-023	Deliver arrivals; list shipments with days late > 0	**Shipment Manager** (`scm_sim.shipments`)	Inventory (receives the delivered stock)	SAD §3.4.3, §4.3.3	TC-SHP-02
SCM-F-030	Report delayed shipments with days late, worst first	**Analytics** (`scm_sim.analytics`)	Shipment Manager (supplies `expected_day` / `actual_day`)	SAD §3.4.4, §4.3.4	TC-ANA-01
SCM-F-031	Report stockouts, low stock and excess with surplus	**Analytics** (`scm_sim.analytics`)	Inventory (supplies the stock snapshot)	SAD §3.4.4, §4.3.4	TC-ANA-01
SCM-F-032	Order metrics per status and fill rate	**Analytics** (`scm_sim.analytics`)	Order Manager (supplies the orders)	SAD §3.4.4, §4.3.4	TC-ANA-02
SCM-F-033	Build the daily report and format it as readable text	**Analytics** (`scm_sim.analytics`)	Simulation Engine and CLI (consume the report)	SAD §3.4.4, §4.3.4	TC-ANA-02
SCM-F-034	Aggregate and format a multi-day run summary, displayed with `--summary`	**Analytics** (`scm_sim.analytics`)	Simulation Engine and CLI (request and display the summary)	SAD §3.4.1, §3.4.4, §4.3.1, §4.3.4	TC-ANA-02, TC-SIM-02
SCM-F-040	Configure a scenario	**Simulation Engine** (`scm_sim.simulator`)	Inventory, Shipment Manager, Order Manager	SAD §3.4.1, §4.3.1	TC-SIM-02
SCM-F-041	Advance one day in the fixed order, keep every report	**Simulation Engine** (`scm_sim.simulator`)	Analytics (builds the report at step 5)	SAD §3.4.1, §4.1	TC-SIM-01
SCM-F-042	Automatic replenishment trimmed to capacity	**Simulation Engine** (`scm_sim.simulator`)	Inventory (`reorder_suggestions`), Shipment Manager (`dispatch`)	SAD §3.4.1, §4.1	TC-SIM-01
SCM-F-043	Command-line interface, invocation and exit behavior	**CLI** (`scm_sim.__main__`)	Simulation Engine (`build_demo`, `run`)	SAD §3.4.1, §4.5	TC-SIM-02
SCM-F-044	Daily metrics CSV export with	**CLI** (`scm_sim.__m`	Simulation Engine	SAD §3.4.1,	TC-SIM-02

Req ID	Requirement short	Primary component	Supporting component(s)	Design section	Test case
	`--csv PATH`	`ain__`)	(`metrics_rows`)	§4.3.1	

3.8.2 Non-Functional Requirements

Req ID	Requirement short	Primary component	Supporting component(s)	Design section	Test case
SCM-NF-001	365-day run and 10-day CLI demo each in ≤ 1.0 s	**Simulation Engine** (`scm_sim.simulator`)	CLI (`scm_sim.__main__`), whole `scm_sim` package	SAD §3.1, §3.6	TC-PERF-01
SCM-NF-002	Same seed and scenario give identical output	**Simulation Engine** (`scm_sim.simulator`)	Shipment Manager (injected PRNG), Analytics (deterministic ordering)	SAD §3.6	TC-REP-01
SCM-NF-003	≥ 90% line coverage, all tests pass before merge	**CI Pipeline** (`.github/workflows/ci.yml`)	All five source modules	SAD §3.4.1, §3.6	TC-COV-01
SCM-NF-004	Runs on Python ≥ 3.11 on Windows, Linux and macOS, standard library only	All five source modules	CI Pipeline (verifies one of the three platforms automatically)	SAD §3.6	TC-COV-01
SCM-NF-005	One documented command; counts and “No issues” in every report section; `--help`	**CLI** (`scm_sim.__main__`)	Analytics (`format_report`)	SAD §4.5	TC-SIM-02

3.8.3 Security Requirements

The two security objectives are architectural in nature and are realised through the components and controls below rather than implemented by any single component.

Req ID	Requirement short	Primary component(s)	Design section	Test case
SCM-SO-1	**Objective:** integrity of simulation data	Inventory, Order Manager, Shipment Manager (all three enforce their invariants)	SAD §3.4.1, §3.4.2, §3.4.3, §4.4	TC-SEC-01
SCM-SO-2	**Objective:** integrity and traceability of the codebase	CI Pipeline and the protected-`main` workflow; Developer (authoring and	SAD §3.9	TC-SEC-02

Req ID	Requirement short	Primary component(s)	Design section	Test case
		reviewing pull requests)		
SCM-SR-001	Validate input on every public operation before mutating state	Inventory, Order Manager, Shipment Manager, Simulation Engine	SAD §4.4	TC-SEC-01
SCM-SR-002	Stock never below 0 or above a registered capacity	Inventory	SAD §3.4.1, §4.4	TC-SEC-01
SCM-SR-003	Guarded order status transitions	Order Manager	SAD §3.4.2, §4.4	TC-SEC-01
SCM-SR-004	Unknown supplier, warehouse or order references raise `KeyError`	Order Manager, Shipment Manager, Simulation Engine	SAD §4.4	TC-SEC-01
SCM-SR-005	`main` is protected: PR, non-author approval, passing `test` check, no force pushes	CI Pipeline and the repository branch-protection settings	SAD §3.9	TC-SEC-02
SCM-SR-006	No secrets; no network calls; no runtime third-party dependencies	All five source modules (`requirements.txt`)	SAD §3.6, §3.9	TC-SEC-02
SCM-SR-007	Returned data is a copy; mutating it cannot change internal state	Inventory (`snapshot`), Analytics (`build_report`)	SAD §3.4.1, §3.4.4	TC-SEC-01

3.8.4 Reverse Coverage (Component to Requirements)

Component	Requirements implemented	Requirements it supports but does not own
Inventory	SCM-F-001, SCM-F-002, SCM-F-003, SCM-F-004	SCM-SR-002, SCM-SR-007
Order Manager	SCM-F-010, SCM-F-011, SCM-F-012, SCM-F-013	SCM-SR-003, SCM-SR-004
Shipment Manager	SCM-F-020, SCM-F-021, SCM-F-022, SCM-F-023	SCM-SR-004, SCM-SR-006
Analytics	SCM-F-030, SCM-F-031, SCM-F-032, SCM-F-033, SCM-F-034	SCM-SR-007, SCM-NF-002, SCM-NF-005
Simulation Engine	SCM-F-040, SCM-F-041, SCM-F-042	SCM-F-044, SCM-SR-001, SCM-NF-001, SCM-NF-002
CLI	SCM-F-043, SCM-F-044	SCM-F-034, SCM-NF-005
CI Pipeline	SCM-NF-003	SCM-SR-005, SCM-SR-006, SCM-NF-004

Every one of the 34 requirement IDs (22 functional, 5 non-functional, 7 security) appears in exactly one “primary component” cell above, and every component in the component diagram of SAD §3.3 implements at least one requirement. No component is orphaned and no requirement is unimplemented.

3.9 Security Architecture

3.9.1 Applicability of STRIDE to This System

STRIDE is normally applied to a networked, multi-user, service-hosted system with an authentication boundary, an authorisation model, network listeners and a persistent data store. The Supply Chain Management Simulator has none of those: it is a single-process Python command-line program that imports only the standard library, keeps all state in memory for the lifetime of one run, opens no sockets, stores nothing to disk and serves no requests. There is therefore no user identity to spoof, no privilege to escalate, no request to flood and no stored record to disclose during a run.

The threats that *are* real for this project are therefore of two kinds, and the security architecture addresses both:

1. **Data-integrity threats at runtime.** The simulation manipulates money-relevant quantities — stock levels, order commitments and shipment arrival days. If bad input could drive stock negative, push a warehouse past its capacity, or produce an impossible status transition, the resulting reports would be confidently wrong. This is the concern of the security objective **SCM-SO-1**, and it is enforced inside the code by validation and invariants.
2. **Supply-chain and process threats during development.** The simulator is not the only place where integrity matters. Anyone who can change the code can change what the simulator reports, and the code is developed by four people on a shared public repository. This is the concern of **SCM-SO-2**, and it is enforced by branch protection, mandatory automated verification and peer review.

Each of the six STRIDE categories is therefore analysed below against the project as built, with the threat re-cast so that it is meaningful for a local simulator and its development pipeline.

3.9.2 STRIDE Analysis

Category	Threat applicable to this system	Mitigation implemented	Related requirement IDs
Spoofing	At runtime there is no authentication surface: no login, no API key and no user account, so an external party cannot impersonate a caller. The equivalent threat in this project is **impersonation in the repository**. A commit attributed to someone other than its true author, or a pull request opened through an account that does not belong to the team member who wrote it, would make the change history untrustworthy and would defeat the traceability promised by SCM-SO-2.	GitHub verifies the email address behind every commit (the repository requires a GitHub-linked email in `git config user.email`), and authorship is displayed against the authenticated GitHub account rather than against free text supplied by the client. No identity is asserted at runtime because no runtime identity exists. Branch protection additionally requires that a change to `main` arrive through a pull request opened by a team member, so an anonymous contribution cannot land directly.	SCM-SO-2, SCM-SR-005
Tampering	Two distinct tampering surfaces exist. **Runtime data tampering:** malformed input reaching a public operation and	*Runtime:* fail-fast validation before any mutation — `ValueError` for invalid arguments (non-positive	SCM-SO-1, SCM-SO-2, SCM-SR-001, SCM-SR-002, SCM-SR-003, SCM-SR-004, SCM-SR-005, SCM-SR-006

Category	Threat applicable to this system	Mitigation implemented	Related requirement IDs
	corrupting the simulation state — stock driven negative, a warehouse exceeding its capacity, an order moved through an illegal status transition, or an order silently fulfilled twice. **Codebase tampering:** a change reaching `main` that has not been reviewed or tested, including a malicious or careless dependency added to the runtime set.	quantity, negative lead time, reliability outside $[0,1]$, non-positive capacity, a scheduled day in the past), `InsufficientStockError` and `CapacityExceededError` for the two stock invariants, and `KeyError` for references to unknown suppliers, warehouses or orders, so that nothing is created implicitly. The overarching invariant is that **any operation which raises leaves state unchanged**. Order status transitions are guarded: a `FULFILLED` order cannot be cancelled, and `FULFILLED` and `CANCELLED` orders are never re-processed. *Codebase:* the `main` branch is protected so that no change can be pushed to it directly — a pull request is required, the required `test` check must pass, the branch must be up to date, and force pushes and deletions are blocked with the rules applied to administrators as well. Peer review by a non-author is the human control.	
Repudiation	A change that cannot be attributed to an author, or a failed check whose result is not retained, would leave the team unable to say who introduced a defect or what state the code passed CI in. Simulation state itself needs no audit trail — it is in-memory and discarded when the run ends.	Every change is traceable to a commit with a recorded author and date, and to the pull request through which it was merged, so a line of code can always be traced back to a person and a discussion. Pull requests retain their review comments and approvals as a permanent record. The GitHub	SCM-SO-2, SCM-SR-005, SCM-NF-003

Category	Threat applicable to this system	Mitigation implemented	Related requirement IDs
		Actions `test` job publishes the result of `pytest -v --cov=scm_sim` on every push to `main` and every pull request, giving a durable, dated record of which tests passed for which commit. This is the control that makes SCM-SO-2's traceability claim checkable rather than aspirational.	
Information disclosure	The runtime surface for disclosure is small but must be stated: there is no data store to dump, no network egress, no log file and no credential store. The real disclosure surface is the **repository**, where a committed token, password or `.env` file would be published to every clone, and the **development machine**, where credentials would be stored insecurely.	The simulator uses only the Python standard library at runtime (`dataclasses`, `enum`, `random`, `argparse`), so it introduces no third-party code that could phone home, and it makes no network calls at all. `requirements.txt` contains only test-time dependencies (`pytest`, `pytest-cov`) and no runtime dependency is installed by an end user, so nothing needs to be trusted or fetched at run time. Secrets are kept out of the repository; repository contents are searched for tokens as part of the manual TC-SEC-02 check. Simulation data is not sensitive because it is entirely synthetic.	SCM-SR-006, SCM-NF-004
Denial of service	There is no server to exhaust, so the realistic equivalent is **resource exhaustion inside a run**: an extreme `--days` value, an oversized quantity, or a single warehouse holding an unbounded number of tracked items could make a run consume unbounded time or memory and appear to hang. A malicious input of	Arguments are validated at the entry point: `--days < 1` is rejected as an argparse usage error with exit code 2 before any simulation is built, so an invalid invocation fails immediately rather than looping. Within the engine, every daily loop is bounded by the number of days requested, non-positive quantities are rejected by `Inventory`,	SCM-SR-001, SCM-NF-001

Category	Threat applicable to this system	Mitigation implemented	Related requirement IDs
	this kind is indistinguishable, from the operator's point of view, from a defect.	and a registered warehouse capacity bounds the stock it can hold. Performance is baselined so that a regression is detectable: a 365-day run completes in approximately 0.02 s against the 1 s budget of SCM-NF-001, measured by TC-PERF-01. There is no authentication, rate limiting or quota layer because there is no service to protect.	
Elevation of privilege	The project has no roles, no permissions model and no authenticated principal, so there is no runtime privilege to escalate. The equivalent threat is **bypassing the review gate on the code itself** — a team member (or anyone with repository access) merging work to `main` without review, or using an administrator account to override the protection rules.	The rules applied to `main` are enforced for administrators as well as for other users, so an administrator cannot bypass them, and force pushes and branch deletion are blocked. A pull request cannot be merged without at least one approval from someone other than its author, and the required `test` check must report success. The verification that these rules are actually in force — attempting a direct push, attempting to merge an unapproved pull request, and inspecting Settings → Branches — is defined as the manual test case TC-SEC-02, so the control is itself verified rather than assumed.	SCM-SO-2, SCM-SR-005

3.9.3 Security Controls Summary

Control	Type	Where enforced	Verified by
Input validation before state mutation	Preventive	`inventory.py`, `orders.py`, `shipments.py`, `simulator.py`	TC-SEC-01, TC-INV-01, TC-ORD-01, TC-SHP-01, TC-SIM-02
Stock invariants (no negative stock, no over-capacity warehouse)	Preventive	`Inventory.add`, `Inventory.remove`	TC-SEC-01, TC-INV-01
Guarded order status	Preventive	`OrderManager.try_fulfil`, `process_open`,	TC-SEC-01, TC-ORD-02

Control	Type	Where enforced	Verified by
transitions		`cancel`	
Rejection of unknown references with `KeyError`	Preventive	`OrderManager.get`, `cancel`, `ShipmentManager.dispatch`, `Simulation`	TC-SEC-01
Returned data returned as a copy	Preventive	`Inventory.snapshot`, `analytics.build_report`	TC-SEC-01, TC-INV-02
Standard library only at runtime; no network calls	Preventive	All five modules; `requirements.txt`	TC-SEC-02
Protected `main` branch rules incl. admins	Preventive	GitHub branch protection	TC-SEC-02
Mandatory peer review (1 non-author approval)	Detective	GitHub pull request review	TC-SEC-02
Automated test and coverage gate on every push and pull request	Detective	`.github/workflows/ci.yml`, job `test`	TC-COV-01, TC-SEC-02
No secrets committed to the repository	Preventive	Repository contents; developer practice	TC-SEC-02

3.9.4 Scope Exclusions

The following are explicitly **not** addressed, because the corresponding features do not exist in the system and their absence is a documented design decision rather than a gap. Stating them prevents a reader from reading an omission as an oversight:

- **Authentication, authorisation, session management and account roles.** The simulator has no user accounts and no login; every invocation runs with the full privileges of the operating-system user who started it.
- **Transport and network security (TLS, API security, firewalling).** The simulator opens no sockets and exchanges no data with another system, so there is nothing to encrypt or authenticate in transit.
- **At-rest encryption, key management and secure deletion.** All simulation state lives in memory for the lifetime of the process and is discarded when it exits.
- **Audit logging and tamper-evident logs.** The event record of a run is the day-by-day `DayLog` and the retained reports, which are diagnostic output rather than a security audit trail, and which disappear with the process.
- **Dependency vulnerability management and SBOM generation.** There are no runtime third-party dependencies to scan; the only dependencies are the test tools, which never execute in production.
- **Physical security and endpoint protection.** Out of scope for a source repository; these are the responsibility of the development environment.

4.1 Design Overview

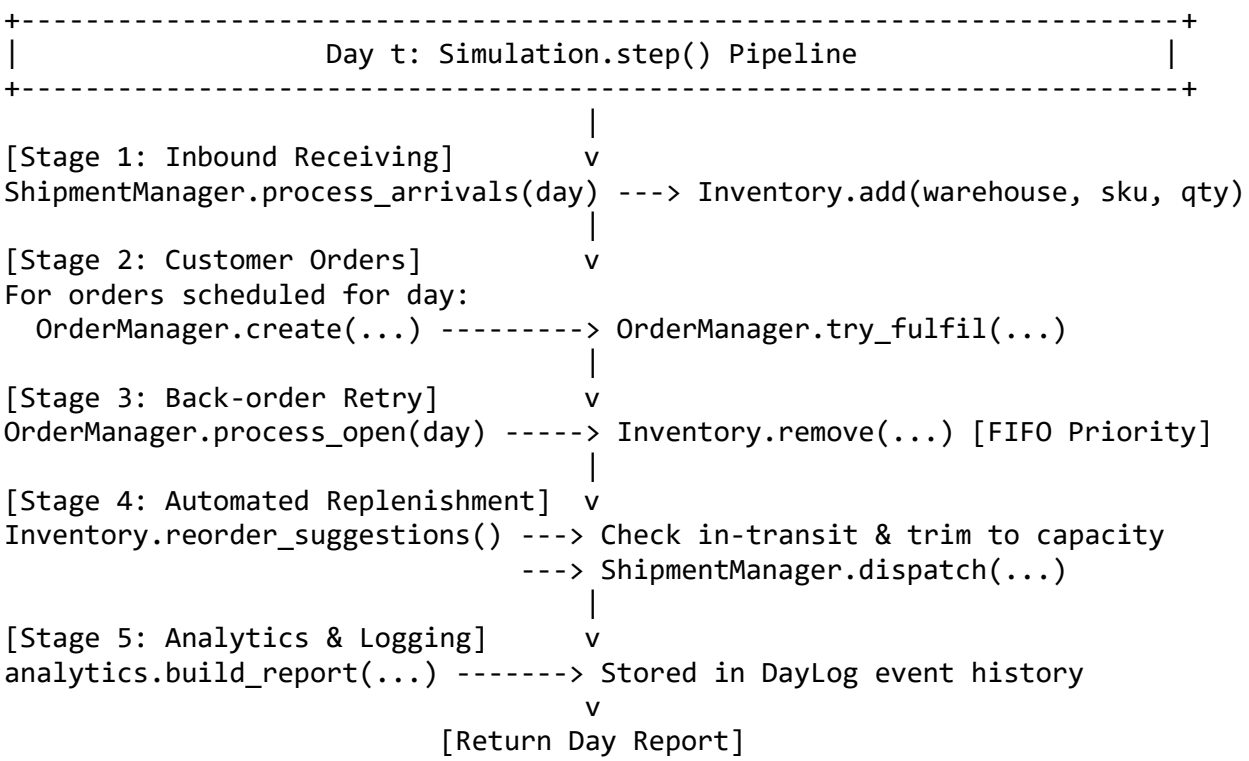
Simulation Paradigm

The Supply Chain Management Simulator employs a **discrete-time, day-by-day stepped simulation engine**. Rather than continuous time or complex asynchronous event queues, the simulation clock advances strictly in discrete integer units of simulated days ($t = 0,1,2, \ldots, N$).

The primary facade orchestrating this lifecycle is `scm_sim.simulator.Simulation`. On each day tick, the simulation executes a deterministic, 5-stage sequence designed to model real-world physical and commercial supply chain flows.

The 5-Stage Daily Execution Sequence (`Simulation.step()`)

Every call to `Simulation.step()` advances the internal day counter by 1 and executes the following operations in a strictly enforced order:



1. **Stage 1: Inbound Receiving (Shipment Arrivals):**
   - The engine triggers `ShipmentManager.process_arrivals(self.day)`.
   - Any shipment in transit whose `arrival_day <= self.day` is delivered.
   - For each arriving shipment, units are credited to warehouse inventory via `Inventory.add(warehouse_id, sku, quantity)`.
   - The shipment status transitions to `DELIVERED` and `actual_day` is set to `self.day`.
   - *Rationale:* Stock must physically arrive at the warehouse before customer orders for the day are evaluated.
2. **Stage 2: Customer Order Processing:**
   - The engine iterates through customer orders pre-scheduled for the current day.
   - For each scheduled order, `OrderManager.create(...)` registers the order with initial status `PENDING`.
   - `OrderManager.try_fulfil(...)` immediately attempts fulfillment:
     - If `Inventory.level(warehouse, sku) >= quantity`: units are removed from inventory via `Inventory.remove()`, status is set to `FULFILLED`, and `fulfilled_day` is recorded.
     - If stock is insufficient: status transitions to `BACKORDERED` with stock level left unchanged (no partial fulfillment).
3. **Stage 3: Back-order Re-evaluation:**
   - The engine calls `OrderManager.process_open(self.day)` to re-evaluate all unresolved `PENDING` and `BACKORDERED` orders.
   - Orders are processed in strict priority sequence: oldest `created_day` first, with ties broken by ascending `order_id` (FIFO fairness).
   - Any newly replenished stock from Stage 1 that was not consumed in Stage 2 is allocated to older backlog orders.
4. **Stage 4: Automated Replenishment Evaluation:**

- The engine queries `Inventory.reorder_suggestions()` to identify tracked items whose stock is at or below their configured `StockPolicy.reorder_point`.
- For each candidate item, the engine verifies that no replenishment shipment is already in transit for that `(warehouse, sku)` pair (preventing redundant double-orders).
- The replenishment quantity (`StockPolicy.reorder_quantity`) is evaluated against warehouse storage limits: if adding the full batch would breach warehouse capacity, the order quantity is trimmed to fit available capacity.
- If positive capacity exists, `ShipmentManager.dispatch(...)` is invoked to schedule the shipment with the designated supplier, computing lead time and stochastic delays.

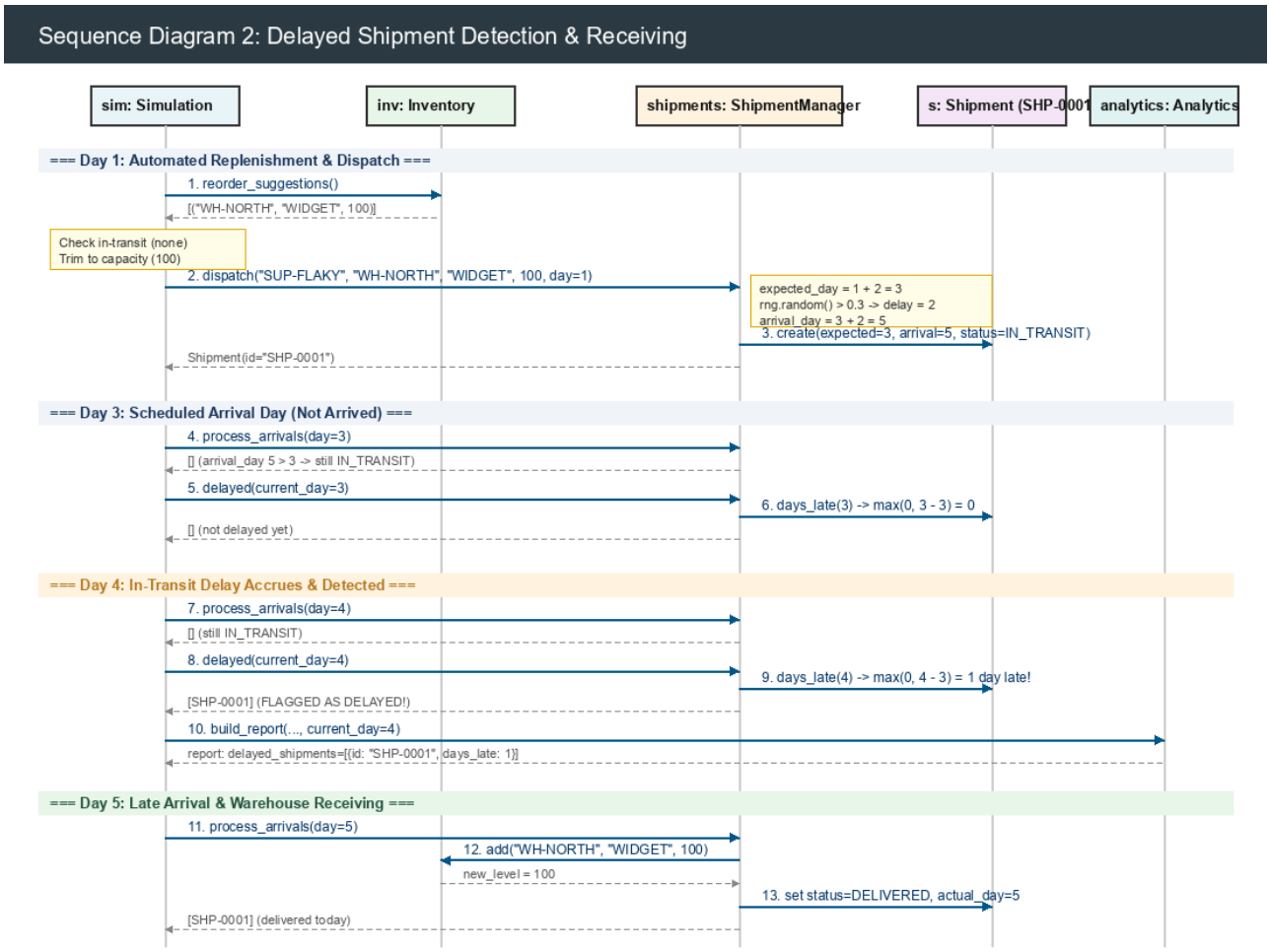
5. **Stage 5: Analytics & Reporting:**
   - The engine queries `Inventory.snapshot()`, `ShipmentManager._shipments.values()`, and `OrderManager._orders.values()`.
   - It invokes the pure function `analytics.build_report(...)`, compiling stockout lists, low-stock alerts, excess inventory levels, late shipment delays, and order fill-rate statistics.
   - The resulting report dictionary is stored in the simulation history (`DayLog`) and returned to the caller.

Data Flow and Subsystem Decoupling

The architecture enforces unidirectional data dependencies to maximize testability and eliminate circular couplings: - `Inventory`: Fundamental state provider; has zero dependencies on other simulation modules. - `OrderManager`: Depends strictly on `Inventory` for stock balance queries and withdrawals. - `ShipmentManager`: Depends strictly on `Inventory` for crediting arrived goods. - `Analytics`: Completely stateless; accepts raw data structures and snapshots, with no dependencies on internal manager state. - `Simulation`: High-level orchestrator mediating interactions between domain managers and the analytics pipeline.

4.2 Sequence Diagram: Delayed Shipment Detection

The following sequence diagram models the lifecycle of a replenishment shipment experiencing stochastic transit delays, detailing the chronological interactions between the `Simulation` engine, `Inventory`, `ShipmentManager`, `Shipment`, and `Analytics` across simulated days 1 through 5.



Sequence diagram: delayed shipment

Step-by-Step Sequence Walkthrough

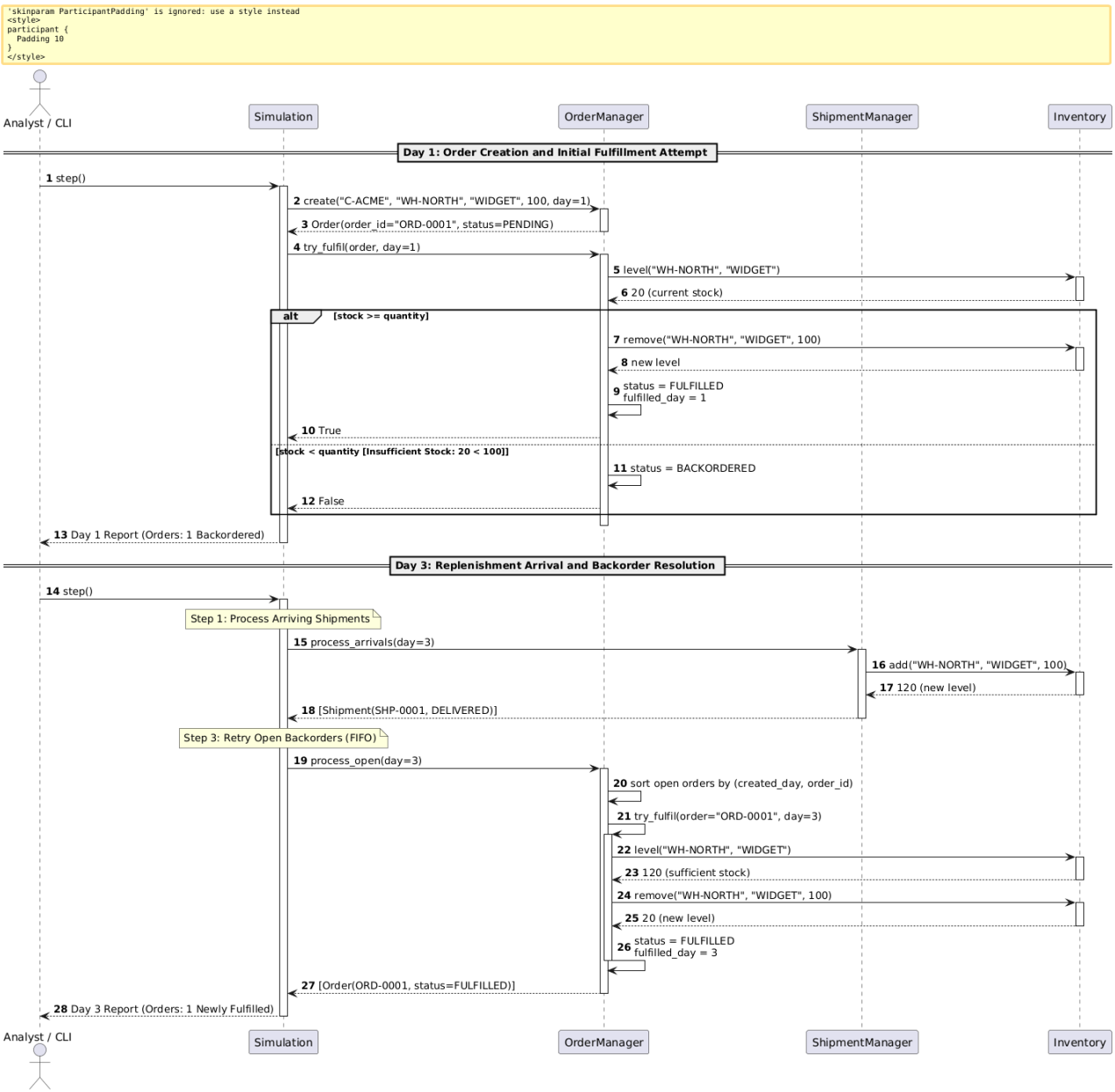
1. **Day 1: Automated Replenishment Evaluation and Dispatch:**
    - In stage 4 of `Simulation.step()` at day 1, the simulation queries `Inventory.reorder_suggestions()`.
    - Inventory reports that SKU `WIDGET` at warehouse `WH-NORTH` is at or below its reorder point, suggesting a reorder quantity of 100 units.
    - The engine checks `ShipmentManager.in_transit()` to verify that no shipment for `(WH-NORTH, WIDGET)` is already pending, and trims the quantity to available warehouse capacity (100 units).
    - The engine calls `ShipmentManager.dispatch(supplier_id="SUP-FLAKY", warehouse_id="WH-NORTH", sku="WIDGET", quantity=100, day=1)`.
    - `ShipmentManager` looks up `SUP-FLAKY` (baseline `lead_time_days = 2`, `reliability = 0.3`).
    - `expected_day` is calculated as `1 + 2 = 3`.
    - Delay sampling: since `delay` is not explicitly provided, the pseudorandom draw from `random.Random` evaluates `rng.random() > 0.3` (True), triggering a delay of `rng.randint(1, 3)` (evaluating to 2 days).
    - `arrival_day` is computed as `expected_day + delay = 3 + 2 = 5`.
    - A `Shipment` instance is instantiated with ID `SHP-0001`, status `IN_TRANSIT`, `expected_day = 3`, and `arrival_day = 5`.
2. **Day 3: Scheduled Expected Arrival (Transit Incomplete):**
    - On day 3, `Simulation.step()` triggers `ShipmentManager.process_arrivals(day=3)`.

- `ShipmentManager` checks all in-transit shipments: `SHP-0001` has `arrival_day = 5`, so `5 <= 3` is `False`. The shipment remains `IN_TRANSIT` and zero inventory is added.
- The engine queries `ShipmentManager.delayed(current_day=3)`.
- `SHP-0001.days_late(3)` evaluates `max(0, current_day - expected_day) = max(0, 3 - 3) = 0`.
- The shipment is not yet classified as delayed, and `delayed()` returns an empty list `[]`.

3. **Day 4: In-Transit Delay Accrual and Exception Detection:**
   - On day 4, `process_arrivals(day=4)` verifies `arrival_day <= 4` (`5 <= 4` is `False`). The shipment remains `IN_TRANSIT`.
   - The engine queries `ShipmentManager.delayed(current_day=4)`.
   - `SHP-0001.days_late(4)` evaluates `max(0, 4 - 3) = 1`. Because `days_late > 0`, the shipment is flagged as an **in-transit delay**.
   - `delayed(4)` returns `[SHP-0001]`.
   - In stage 5, `analytics.build_report(...)` invokes `find_delayed_shipments(shipments, current_day=4)`, returning a record with `shipment_id="SHP-0001"`, `sku="WIDGET"`, `warehouse_id="WH-NORTH"`, and `days_late=1`.
   - The daily simulation report alerts the Supply Chain Analyst to the delayed goods.
4. **Day 5: Physical Inbound Receiving and Post-Delivery Historical Tracking:**
   - On day 5, `process_arrivals(day=5)` checks `SHP-0001`: `arrival_day <= 5` (`5 <= 5` is `True`).
   - `ShipmentManager` invokes `Inventory.add(warehouse="WH-NORTH", sku="WIDGET", qty=100)`, increasing warehouse stock to 100 units.
   - `SHP-0001` status transitions to `ShipmentStatus.DELIVERED`, and `actual_day` is recorded as `5`.
   - When subsequent delay checks run, `SHP-0001.delay_days` and `SHP-0001.days_late(5)` evaluate to `actual_day - expected_day = 5 - 3 = 2`.
   - The delivered shipment remains accurately tracked as **delivered late** in the simulation history for root-cause supplier performance analysis.

4.2 Sequence Diagram 1: Order Fulfilment and Back-Order Flow

This section details the dynamic execution flow for customer order creation, initial stock inspection, backorder transition, and subsequent resolution following replenishment delivery.



Sequence diagram: order fulfilment

Step-by-Step Flow Explanation

The sequence diagram illustrates the lifecycle of customer order `ORD-0001` interacting with `Simulation`, `OrderManager`, `ShipmentManager`, and `Inventory` across two discrete simulation days:

1. **Step Trigger (Day 1):** The user or CLI invokes `Simulation.step()`, advancing the discrete time counter to Day 1.
2. **Order Instantiation:** The simulation orchestrator retrieves orders scheduled for Day 1 and calls `OrderManager.create(customer_id="C-ACME", warehouse_id="WH-NORTH", sku="WIDGET", quantity=100, day=1)`. The manager validates that quantity > 0, issues sequential identifier `"ORD-0001"`, stores the order internally with status `PENDING`, and returns the `Order` instance.
3. **Immediate Fulfillment Check:** The simulation invokes `OrderManager.try_fulfil(order, day=1)`. The manager queries `Inventory.level("WH-NORTH", "WIDGET")`, which returns current available stock of 20 units.
4. **Shortage & Backordering:** Because the available stock (20) is strictly less than the requested quantity (100), the all-or-nothing fulfillment rule triggers:

- No inventory is deducted (stock remains at 20).
- The order status transitions from PENDING to BACKORDERED.
- `try_fulfil()` returns `False`.

5. **Day 1 Closure:** `Simulation` compiles Day 1 analytics, reflecting one active backorder, and outputs the day log.
6. **Step Trigger (Day 3):** Two days later, `Simulation.step()` is invoked for Day 3.
7. **Shipment Arrival (Step 1 of Daily Loop):** `Simulation` invokes `ShipmentManager.process_arrivals(day=3)`. Shipment `SHP-0001` (carrying 100 units of `WIDGET` dispatched from supplier `SUP-RELIABLE`) arrives. The shipment manager calls `Inventory.add("WH-NORTH", "WIDGET", 100)`, increasing total on-hand stock to 120 units, marks the shipment DELIVERED, and records `actual_day = 3`.
8. **FIFO Backorder Retry (Step 3 of Daily Loop):** Later in the Day 3 cycle, `Simulation` invokes `OrderManager.process_open(day=3)`. The manager collects all open orders (PENDING or BACKORDERED) and sorts them strictly oldest-first by `(created_day, order_id)`.
9. **Successful Fulfillment:** `ORD-0001` is evaluated first. `OrderManager` queries `Inventory.level("WH-NORTH", "WIDGET")`, which now returns 120 units. Because stock (120) ≥ quantity (100):
   - `Inventory.remove("WH-NORTH", "WIDGET", 100)` is called, reducing stock to 20 units.
   - The order status transitions to `FULFILLED` and `fulfilled_day = 3` is recorded.
   - `ORD-0001` is appended to the list of newly fulfilled orders returned to the simulation engine.
10. **Day 3 Closure:** The simulation records the fulfillment event in the Day 3 analytics report and updates the customer fill rate metric.

4.3.4 Analytics API (`scm_sim.analytics`)

The Analytics component exports nine public module-level functions and no classes. Every function is **pure**: it reads only its arguments, allocates and returns new containers, and never mutates an argument or module-level state. It imports `Any`, `Mapping` and `Optional` from the standard library and depends on no other `scm_sim` component; its coupling is through caller-supplied data (SAD §3.4.4).

Input and error behaviour. Analytics does not validate its inputs. Its functions do not mutate state; malformed caller data can still cause normal Python exceptions (for example, `AttributeError` for a missing object attribute or `KeyError` for a missing report key). Negative threshold values are accepted as supplied. Domain components validate operational input (SCM-SR-001); Analytics is a read-only reporting layer.

#	Function	Purpose
1	`find_delayed_shipments(shipments, current_day)`	Detect shipments that are later than their expected arrival day.
2	`find_stockouts(stock)`	Detect items whose quantity is exactly zero.
3	`find_low_stock(stock, reorder_point=20, policies=None)`	Detect items at or below the item policy's reorder point, or the fallback threshold, but above zero.
4	`find_excess(stock, max_level=500, policies=None)`	Detect items above the item policy's max level, or the fallback threshold, with the surplus quantity.
5	`order_metrics(orders)`	Count orders per status and compute the fill rate.
6	`build_report(stock, shipments, orders, current_day,`	Compose findings 1–5 into one daily report dictionary.

#	Function	Purpose
	`reorder_point=20, max_level=500, policies=None)`	
7	`format_report(report)`	Render a report dictionary as readable multi-line text.
8	`summarize_run(history)`	Aggregate report counts and final fill rate across a run.
9	`format_run_summary(summary)`	Render a run-summary dictionary as readable multi-line text.

---

1. find_delayed_shipments(shipments, current_day)

- **Signature:** `find_delayed_shipments(shipments: list[Any], current_day: int) -> list[dict[str, Any]]`
- **Parameters:**
  - `shipments` (`list`): The shipments to inspect. Each element need only expose the attributes `shipment_id` (`str`), `sku` (`str`), `warehouse_id` (`str`), `expected_day` (`int`) and `actual_day` (`int` or `None`). Any object with those attributes works, which is why the tests pass stand-ins built with `types.SimpleNamespace`. Passing the result of `ShipmentManager.in_transit()` plus the delivered shipments covers the whole run.
  - `current_day` (`int`): The current simulation day, used as the reference point for every shipment that has not yet been delivered.
- **Returns:** A `list` of `dict`, one per late shipment, each with exactly the keys `shipment_id`, `sku`, `warehouse_id` and `days_late`. A shipment is included **if and only if** `days_late > 0`, where `days_late` is computed as `actual_day - expected_day` for a delivered shipment and `current_day - expected_day` for one still in transit. The list is sorted by `days_late` descending, with `shipment_id` ascending as a deterministic tie-break. Returns an empty list when nothing is late.
- **Raises:** None.
- **Example:**

```python
from scm_sim.analytics import find_delayed_shipments

delayed = find_delayed_shipments(shipments, current_day=4)
for d in delayed:
    print(f"{d['shipment_id']}: {d['sku']} to {d['warehouse_id']}, "
          f"{d['days_late']} day(s) late")
# SHP-0001: WIDGET to WH-NORTH, 1 day(s) late
```

- **Notes:** A shipment arriving exactly on its expected day, or before it, is never reported. The delivered branch selects on `actual_day is not None` rather than on truthiness, so a shipment that arrived on day 0 is correctly treated as delivered rather than as perpetually in transit.

---

2. find_stockouts(stock)

- **Signature:** `find_stockouts(stock: dict[tuple[str, str], int]) -> list[tuple[str, str]]`
- **Parameters:**
  - `stock` (`dict`): The stock snapshot keyed by `(warehouse_id, sku)`. Pass the result of `Inventory.snapshot()`, which is an independent copy.
- **Returns:** A `list` of `(warehouse_id, sku)` tuples for every item whose quantity is **exactly** 0, sorted ascending (by `warehouse_id`, then `sku`). An item with any positive quantity — including 1 — is not a stockout and is not returned. Returns an empty list for an empty snapshot.

- **Raises:** None.
- **Example:**

```
from scm_sim.analytics import find_stockouts

find_stockouts({("WH-NORTH", "WIDGET"): 0, ("WH-NORTH", "GADGET"): 1})
# [('WH-NORTH', 'WIDGET')]
```

---

3. find_low_stock(stock, reorder_point=20, policies=None)

- **Signature:** `find_low_stock(stock: dict[tuple[str, str], int], reorder_point: int = 20, policies: Optional[Mapping[tuple[str, str], Any]] = None) -> list[tuple[str, str, int]]`
- **Parameters:**
  - `stock` (`dict`): The stock snapshot keyed by `(warehouse_id, sku)`.
  - `reorder_point` (`int`, default `20`): The inclusive upper bound of the low-stock band. The default matches the default `StockPolicy(reorder_point=20)`.
- `policies` (`Mapping`, default `None`): Optional map from `(warehouse_id, sku)` to an object exposing `reorder_point`; when an item is present, this value takes precedence over the fallback `reorder_point`.
- **Returns:** A `list` of `(warehouse_id, sku, qty)` tuples for every item where $0 < qty \leq$ the applicable threshold (item policy when present, otherwise `reorder_point`), sorted ascending by warehouse and SKU. A quantity of 0 is **excluded** — that is a stockout, reported by `find_stockouts`, not low stock. A quantity equal to the threshold is **included**.
- **Raises:** None.
- **Example:**

```
from scm_sim.analytics import find_low_stock

find_low_stock({("WH-NORTH", "GADGET"): 1, ("WH-NORTH", "GIZMO"): 20,
                ("WH-NORTH", "BOLT"): 21})
# [('WH-NORTH', 'GADGET', 1), ('WH-NORTH', 'GIZMO', 20)]

find_low_stock({("WH-NORTH", "BOLT"): 21}, reorder_point=21)
# [('WH-NORTH', 'BOLT', 21)]
```

---

4. find_excess(stock, max_level=500, policies=None)

- **Signature:** `find_excess(stock: dict[tuple[str, str], int], max_level: int = 500, policies: Optional[Mapping[tuple[str, str], Any]] = None) -> list[tuple[str, str, int, int]]`
- **Parameters:**
  - `stock` (`dict`): The stock snapshot keyed by `(warehouse_id, sku)`.
  - `max_level` (`int`, default `500`): The exclusive lower bound of the excess band. The default matches the default `StockPolicy(max_level=500)`.
- `policies` (`Mapping`, default `None`): Optional map from `(warehouse_id, sku)` to an object exposing `max_level`; when an item is present, this value takes precedence over the fallback `max_level`.
- **Returns:** A `list` of `(warehouse_id, sku, qty, surplus)` tuples for every item where `qty` exceeds its applicable threshold, sorted ascending by warehouse and SKU. `surplus` is `qty − applicable threshold`. A quantity equal to the threshold is **excluded**.
- **Raises:** None.
- **Example:**

```python
from scm_sim.analytics import find_excess

find_excess({("WH-NORTH", "GIZMO"): 500, ("WH-NORTH", "BOLT"): 501})
# [('WH-NORTH', 'BOLT', 501, 1)]

find_excess({("WH-NORTH", "WIDGET"): 120}, max_level=100)
# [('WH-NORTH', 'WIDGET', 120, 20)]
```

---

5. order_metrics(orders)

- **Signature:** `order_metrics(orders: list[Any]) -> dict[str, Any]`
- **Parameters:**
  - `orders` (`list`): The orders to aggregate. Each element need only expose a `status` attribute. Pass the list returned by `OrderManager.by_status(...)`, or the manager's full order list.
- **Returns:** A `dict` with exactly the keys `total`, `pending`, `fulfilled`, `backordered`, `cancelled` and `fill_rate`. `total` is the length of the supplied list; the four status counts correspond to `OrderStatus.PENDING`, `FULFILLED`, `BACKORDERED` and `CANCELLED`. `fill_rate` is round(fulfilled ÷ (total − cancelled), 2), or `0.0` when `total - cancelled` is not greater than 0. Status values are accepted as `OrderStatus` enumeration members, as plain strings, or as a mix of the two within one list; a status value outside the four known names is ignored rather than raising.
- **Raises:** None.
- **Example:**

```python
from scm_sim.orders import OrderStatus
from scm_sim.analytics import order_metrics

m = order_metrics(orders)   # FULFILLED, FULFILLED, PENDING, CANCELLED
m["total"]        # 4
m["fulfilled"]    # 2
m["fill_rate"]    # 0.67  -> round(2 / (4 - 1), 2)

order_metrics([])["fill_rate"]   # 0.0
```

---

6. build_report(stock, shipments, orders, current_day, reorder_point=20, max_level=500, policies=None)

- **Signature:** `build_report(stock: dict[tuple[str, str], int], shipments: list[Any], orders: list[Any], current_day: int, reorder_point: int = 20, max_level: int = 500, policies: Optional[Mapping[tuple[str, str], Any]] = None) -> dict[str, Any]`
- **Parameters:**
  - `stock` (`dict`): The stock snapshot, forwarded unchanged to `find_stockouts`, `find_low_stock` and `find_excess`.
  - `shipments` (`list`): The shipments to inspect, forwarded to `find_delayed_shipments`.
  - `orders` (`list`): The orders to aggregate, forwarded to `order_metrics`.
  - `current_day` (`int`): The simulation day. It is stored under `day` **and** used as the reference point for in-transit lateness.
  - `reorder_point` (`int`, default `20`): Forwarded to `find_low_stock`.
  - `max_level` (`int`, default `500`): Forwarded to `find_excess`.
  - `policies` (`Mapping`, default `None`): Optional mapping of item keys to stock-policy-like objects; each item's own thresholds override the fallback arguments.
- **Returns:** A new `dict` with exactly six keys: `day` (the `current_day` as given), `delayed_shipments`, `stockouts`, `low_stock`, `excess` and `orders` (the dictionary from `order_metrics`). Because the

dictionary is freshly allocated on every call, the caller — the Simulation Engine — may retain every daily report for the whole run and compare them later.

- **Raises:** None.
- **Example:**

```
from scm_sim.analytics import build_report

report = build_report(inv.snapshot(), in_transit + delivered, orders,
current_day=4)
sorted(report.keys())
# ['day', 'delayed_shipments', 'excess', 'low_stock', 'orders', 'stockouts']
report["day"]            # 4
```

- **Notes:** This is the only daily-report entry point the Simulation Engine calls directly; it supplies the inventory policies for tracked items and uses the global thresholds only as fallbacks. The five classification/aggregation functions are also used directly in tests and ad-hoc analysis.

---

7. format_report(report)

- **Signature:** `format_report(report: dict[str, Any]) -> str`
- **Parameters:**
  - `report` (`dict`): A report dictionary as returned by `build_report`. All six keys must be present; a missing key raises `KeyError`, which indicates a caller error rather than invalid simulation data.
- **Returns:** A `str` containing newline-separated lines. The first line is the header `=== Supply Chain Report: Day N ===`. Five sections follow, each preceded by a blank line and headed `Section name (count)`, in this fixed order: `Delayed shipments`, `Stockouts`, `Low stock`, `Excess` and `Orders`. When a section is empty the line `No issues` is printed in place of item listings; when it is non-empty one `- …` line is printed per item. The `Orders` section always prints two lines — a status breakdown and `fill rate: <value>`.
- **Raises:** None for a well-formed report; `KeyError` if a required key is absent.
- **Example:**

```
from scm_sim.analytics import build_report, format_report

print(format_report(build_report({}, [], [], current_day=1)))
# === Supply Chain Report: Day 1 ===
#
# Delayed shipments (0)
#   No issues
#
# Stockouts (0)
#   No issues
#
# Low stock (0)
#   No issues
#
# Excess (0)
#   No issues
#
# Orders (0)
#   pending 0, fulfilled 0, backordered 0, cancelled 0
#   fill rate: 0.0
```

8. summarize_run(history)

- **Signature:** `summarize_run(history: list[dict[str, Any]]) -> dict[str, Any]`

- **Parameters:** `history` is the ordered list of daily report dictionaries, normally `Simulation.history`.
- **Returns:** A new dictionary with `days`, `days_with_delays`, `days_with_stockouts`, `days_with_low_stock`, `days_with_excess`, `peak_delayed` and `final_fill_rate`. Day counters count reports containing one or more matching items; `peak_delayed` is the largest delayed-shipment list length on any day. For empty history, counts and `peak_delayed` are zero and `final_fill_rate` is `0.0`.
- **Raises:** None for a well-formed history; malformed report dictionaries may raise the corresponding Python key/type errors.
- **Limitations:** This is a set of run-wide counts and final/peak values, not a time series or trend analysis. It does not calculate average delay or time-to-resolution for delayed shipments or inventory issues.

9. `format_run_summary(summary)`

- **Signature:** `format_run_summary(summary: dict[str, Any]) -> str`
- **Parameters:** A summary dictionary returned by `summarize_run`.
- **Returns:** A multi-line string headed `=== Run Summary ===` and displaying each of the seven summary fields.
- **Raises:** `KeyError` if a required summary key is absent.

7.1 Data Formats Consumed and Produced

Argument	Expected shape	Example	Produced by
`stock`	`dict[{(warehouse_id, sku): int}]`	`{("WH-NORTH", "WIDGET"): 0}`	`Inventory.snapshot()`
`shipments`	`list` of objects with `shipment_id`, `sku`, `warehouse_id`, `expected_day`, `actual_day`	`Shipment(id="SHP-0001", expected_day=3, actual_day=None)`	`ShipmentManager.in_transit()` / delivered shipments
`orders`	`list` of objects with `status` (`OrderStatus` or `str`)	`Order(id="ORD-0001", status=OrderStatus.FULFILLED)`	`OrderManager.by_status(...)`
`current_day`	`int`, the simulation day	`4`	`Simulation` day counter
`policies`	Optional mapping `{(warehouse_id, sku): object}` exposing `reorder_point` and `max_level`	`{("WH-NORTH", "WIDGET"): StockPolicy(...)}`	`Simulation._build_report`
`report`	`dict` with the six report keys	see `build_report`	`build_report`
`history`	Ordered `list` of daily report dictionaries	`Simulation.history`	`Simulation`
`summary`	Dictionary with run-level count and final-fill-rate keys	`summarize_run(history)`	`summarize_run`

4.3 API Design

This section defines the public interface of every code component. For each call it gives the **signature, parameters, return value, exceptions raised and an example**. All examples use made-up data and were

executed against the code at commit `b3702ef`; the output shown is the real output. Members whose names start with _ are private and not part of the API.

4.3.1 Inventory, Simulation Engine and CLI

Inventory API (`scm_sim.inventory`)

Conventions: `warehouse` and `sku` are non-empty strings such as `"WH-NORTH"` and `"WIDGET"`; quantities are integers. Every method that raises leaves the inventory unchanged.

`StockPolicy`

Field	Content
Signature	`StockPolicy(reorder_point: int = 20, reorder_quantity: int = 100, max_level: int = 500)`, frozen dataclass
Parameters	`reorder_point`: reorder when the level is at or below this value. `reorder_quantity`: units to request per reorder. `max_level`: levels above this are excess.
Returns	An immutable policy object
Raises	`ValueError` if any value is negative, or if `max_level < reorder_point`

```
>>> from scm_sim.inventory import Inventory, StockPolicy,
InsufficientStockError, CapacityExceededError
>>> StockPolicy()
StockPolicy(reorder_point=20, reorder_quantity=100, max_level=500)
>>> StockPolicy(reorder_point=30, reorder_quantity=150, max_level=20)
Traceback (most recent call last):
ValueError: max_level must be >= reorder_point
```

`Inventory()`

Field	Content
Signature	`Inventory()`
Parameters	None
Returns	A new, empty inventory: no stock, no registered warehouses, no policies
Raises	–

`level`

Field	Content
Signature	`level(warehouse: str, sku: str) -> int`
Parameters	`warehouse`, `sku`: the item to look up
Returns	Current quantity; `0` if the item has never been stocked
Raises	–

```
>>> inv = Inventory()
>>> inv.level("WH-NORTH", "WIDGET")
0
```

`register_warehouse`

Field	Content

Field	Content
Signature	`register_warehouse(warehouse: str, capacity: int) -> None`
Parameters	`warehouse`: warehouse ID. `capacity`: maximum total units across all SKUs (> 0). Re-registering replaces the capacity.
Returns	None
Raises	`ValueError` if `capacity <= 0`

```
>>> inv.register_warehouse("WH-NORTH", 1200)
>>> inv.register_warehouse("WH-SOUTH", 0)
Traceback (most recent call last):
ValueError: capacity must be positive
```

add

Field	Content
Signature	`add(warehouse: str, sku: str, qty: int) -> int`
Parameters	`qty`: units to add (> 0)
Returns	The new level of the item
Raises	`ValueError` if `qty <= 0`; `CapacityExceededError` if the warehouse is registered and its total would exceed capacity

```
>>> inv.add("WH-NORTH", "WIDGET", 100)
100
>>> inv.add("WH-NORTH", "GIZMO", 1100)
1100
>>> inv.add("WH-NORTH", "GIZMO", 1)
Traceback (most recent call last):
scm_sim.inventory.CapacityExceededError: adding 1 GIZMO would exceed capacity
1200 at WH-NORTH
>>> inv.total_in_warehouse("WH-NORTH")
1200
```

remove

Field	Content
Signature	`remove(warehouse: str, sku: str, qty: int) -> int`
Parameters	`qty`: units to remove (> 0)
Returns	The new level of the item
Raises	`ValueError` if `qty <= 0`; `InsufficientStockError` if `qty` is greater than the current level

```
>>> inv.remove("WH-NORTH", "WIDGET", 30)
70
>>> inv.remove("WH-NORTH", "WIDGET", 71)
Traceback (most recent call last):
scm_sim.inventory.InsufficientStockError: not enough WIDGET at WH-NORTH
>>> inv.level("WH-NORTH", "WIDGET")
70
```

is_stockout

Field	Content
Signature	`is_stockout(warehouse: str, sku: str) -> bool`
Parameters	The item to check
Returns	`True` if the level is exactly 0 (including never-stocked items)
Raises	–

```
>>> inv.is_stockout("WH-NORTH", "WIDGET"), inv.is_stockout("WH-SOUTH", "GADGET")
(False, True)
```

`total_in_warehouse`

Field	Content
Signature	`total_in_warehouse(warehouse: str) -> int`
Parameters	`warehouse`: warehouse ID
Returns	Sum of the levels of all SKUs in that warehouse
Raises	–

```
>>> inv.total_in_warehouse("WH-NORTH")
1170
```

`set_policy` / `get_policy`

Field	Content
Signature	`set_policy(warehouse: str, sku: str, policy: StockPolicy) -> None`; `get_policy(warehouse: str, sku: str) -> StockPolicy`
Parameters	`policy`: the policy for that item; replaces any previous one
Returns	`None` / the item's policy, or `StockPolicy()` if none was set
Raises	–

```
>>> inv.set_policy("WH-NORTH", "WIDGET", StockPolicy(reorder_point=80, reorder_quantity=150, max_level=500))
>>> inv.get_policy("WH-NORTH", "WIDGET").reorder_quantity
150
>>> inv.get_policy("WH-SOUTH", "GADGET")
StockPolicy(reorder_point=20, reorder_quantity=100, max_level=500)
```

`needs_reorder` / `is_excess`

Field	Content
Signature	`needs_reorder(warehouse: str, sku: str) -> bool`; `is_excess(warehouse: str, sku: str) -> bool`
Parameters	The item to check
Returns	`needs_reorder`: `level <= policy.reorder_point`. `is_excess`: `level > policy.max_level`.
Raises	–

```
>>> inv.needs_reorder("WH-NORTH", "WIDGET")
True
>>> inv.is_excess("WH-NORTH", "GIZMO")
True
```

reorder_suggestions

Field	Content
Signature	`reorder_suggestions() -> list[tuple[str, str, int]]`
Parameters	None
Returns	`(warehouse, sku, reorder_quantity)` for every tracked item (ever stocked, or with a policy) that needs a reorder, sorted by warehouse then SKU
Raises	–

```
>>> inv.reorder_suggestions()
[('WH-NORTH', 'WIDGET', 150)]
```

snapshot

Field	Content
Signature	`snapshot() -> dict[tuple[str, str], int]`
Parameters	None
Returns	A new dictionary `{(warehouse, sku): qty}` of all tracked stock; changing it does not change the inventory
Raises	–

```
>>> snap = inv.snapshot()
>>> snap
{('WH-NORTH', 'WIDGET'): 70, ('WH-NORTH', 'GIZMO'): 1100}
>>> snap[("WH-NORTH", "WIDGET")] = 0
>>> inv.level("WH-NORTH", "WIDGET")
70
```

Simulation Engine API (`scm_sim.simulator`)

Conventions: day 0 is set-up time and the first `step()` simulates day 1. Set-up methods may also be called between steps.

Simulation

Field	Content
Signature	`Simulation(seed: int = 42, reorder_point: int = 20, max_level: int = 500)`
Parameters	`seed`: seed of the `random.Random` that draws shipment delays. `reorder_point`, `max_level`: the global low-stock and excess thresholds used in Analytics reports (replenishment uses per-item policies).
Returns	A simulation at `day == 0` with an empty `Inventory`, `OrderManager` and `ShipmentManager` (exposed as `inventory`,

Field	Content
	`orders`, `shipments`), and empty `history` and `logs`
Raises	–

```
>>> from scm_sim.simulator import Simulation, build_demo, describe_day
>>> sim = Simulation(seed=42)
>>> sim.day, sim.history
(0, [])
```

`add_supplier`

Field	Content
Signature	`add_supplier(supplier_id: str, name: str, lead_time_days: int = 2, reliability: float = 1.0) -> Supplier`
Parameters	`lead_time_days`: base delivery time (≥ 0). `reliability`: probability of on-time arrival, 0.0–1.0.
Returns	The registered `Supplier`
Raises	`ValueError` if `lead_time_days < 0`, `reliability` is outside [0, 1], or the ID is already registered

```
>>> sim.add_supplier("SUP-RELIABLE", "Reliable Parts Ltd", lead_time_days=2,
reliability=1.0)
Supplier(supplier_id='SUP-RELIABLE', name='Reliable Parts Ltd',
lead_time_days=2, reliability=1.0)
```

`add_warehouse`

Field	Content
Signature	`add_warehouse(warehouse_id: str, capacity: int) -> None`
Parameters	`capacity`: total units across all SKUs (> 0)
Returns	`None`
Raises	`ValueError` if `capacity <= 0`

```
>>> sim.add_warehouse("WH-NORTH", capacity=1200)
```

`set_policy`

Field	Content
Signature	`set_policy(warehouse_id: str, sku: str, supplier_id: str, policy: StockPolicy | None = None) -> None`
Parameters	`supplier_id`: the one supplier that replenishes this item. `policy`: thresholds; `None` means `StockPolicy()`.
Returns	`None`
Raises	`KeyError` if the warehouse or the supplier has not been added

```
>>> sim.set_policy("WH-NORTH", "WIDGET", "SUP-RELIABLE", StockPolicy(20, 150,
500))
>>> sim.set_policy("WH-NORTH", "WIDGET", "SUP-UNKNOWN")
```

```
Traceback (most recent call last):
KeyError: "Unknown supplier: 'SUP-UNKNOWN'"
```

add_stock

Field	Content
Signature	`add_stock(warehouse_id: str, sku: str, quantity: int) -> int`
Parameters	`quantity`: units to place in stock (> 0)
Returns	The new level of the item
Raises	`KeyError` if the warehouse has not been added; `ValueError` if `quantity <= 0`; `CapacityExceededError` if capacity would be exceeded

```
>>> sim.add_stock("WH-NORTH", "WIDGET", 30)
30
```

schedule_order

Field	Content
Signature	`schedule_order(day: int, customer_id: str, warehouse_id: str, sku: str, quantity: int) -> None`
Parameters	`day`: the future day on which the order is created (> current `day`). `quantity`: units ordered (> 0).
Returns	`None`; the demand is queued until that day
Raises	`ValueError` if `day <= sim.day` or `quantity <= 0`; `KeyError` if the warehouse has not been added

```
>>> sim.schedule_order(1, "C-ACME", "WH-NORTH", "WIDGET", 25)
>>> sim.schedule_order(2, "C-BOLT", "WH-NORTH", "WIDGET", 40)
>>> sim.schedule_order(1, "C-ACME", "WH-EAST", "WIDGET", 5)
Traceback (most recent call last):
KeyError: "Unknown warehouse: 'WH-EAST'"
```

step

Field	Content
Signature	`step() -> dict`
Parameters	None
Returns	The Analytics report for the new day, with keys `day`, `delayed_shipments`, `stockouts`, `low_stock`, `excess`, `orders`. The report is also appended to `history`, and a `DayLog` to `logs`.
Raises	– (validation happens when the scenario is configured)
Behaviour	Increments `day`, then: (1) `process_arrivals`; (2) `process_open` for older back-orders; (3) create and `try_fulfil` today's scheduled orders; (4) replenish each item needing a reorder with nothing in transit, trimmed to capacity; (5) `build_report`.

```
>>> report = sim.step()
>>> report["day"], report["low_stock"], report["orders"]["fulfilled"]
(1, [('WH-NORTH', 'WIDGET', 5)], 1)
```

```
>>> [(s.shipment_id, s.sku, s.quantity, s.expected_day) for s in
sim.logs[0].reorders]
[('SHP-0001', 'WIDGET', 150, 3)]
```

run

Field	Content
Signature	`run(days: int) -> list[dict]`
Parameters	`days`: number of further days to simulate (≥ 0)
Returns	The reports of those days, in order
Raises	`ValueError` if `days < 0`

```
>>> [r["day"] for r in sim.run(2)]
[2, 3]
>>> sim.orders.get("ORD-0002").status.value, sim.orders.get("ORD-
0002").fulfilled_day
('FULFILLED', 3)
```

final_summary

Field	Content
Signature	`final_summary() -> str`
Parameters	None
Returns	The latest report formatted by Analytics `format_report`; if no day has been simulated, a report for day 0
Raises	–

```
>>> print(sim.final_summary().splitlines()[0])
=== Supply Chain Report: Day 3 ===
```

all_orders

Field	Content
Signature	`all_orders() -> list[Order]`
Parameters	None
Returns	Every order created so far, in order-ID order
Raises	–

```
>>> [(o.order_id, o.customer_id, o.status.value) for o in sim.all_orders()]
[('ORD-0001', 'C-ACME', 'FULFILLED'), ('ORD-0002', 'C-BOLT', 'FULFILLED')]
```

build_demo

Field	Content
Signature	`build_demo(seed: int = 42) -> Simulation`
Parameters	`seed`: passed to `Simulation`
Returns	A ready-to-run scenario at day 0: suppliers `SUP-RELIABLE` (1.0) and `SUP-FLAKY` (0.3), warehouses `WH-NORTH` (1200) and `WH-SOUTH` (500), SKUs `WIDGET`, `GADGET` and `GIZMO`, and 14 orders over days 1–10
Raises	–

```
>>> demo = build_demo(seed=42)
>>> reports = demo.run(10)
>>> reports[-1]["delayed_shipments"][0]
{'shipment_id': 'SHP-0001', 'sku': 'GADGET', 'warehouse_id': 'WH-SOUTH',
'days_late': 1}
>>> reports[-1]["excess"]
[('WH-NORTH', 'GIZMO', 600, 100)]
```

describe_day

Field	Content
Signature	`describe_day(log: DayLog, report: dict) -> str`
Parameters	One day's `DayLog` and its report
Returns	A multi-line, human-readable summary of the day, ending with a `flags:` line
Raises	–

```
>>> print(describe_day(demo.logs[2], reports[2]))
Day 3:
  fulfilled ORD-0006: 45 WIDGET from WH-NORTH
  BACKORDER ORD-0005: 30 GADGET from WH-SOUTH
  flags: 0 delayed, 1 stockout, 0 low, 1 excess | fill rate 0.83
```

metrics_rows

Field	Content
Signature	`metrics_rows(history: list[dict]) -> list[dict]`
Parameters	Ordered daily report dictionaries, normally `Simulation.history`
Returns	One flat row per report with `day`, `delayed`, `stockouts`, `low_stock`, `excess`, `pending`, `fulfilled`, `backordered`, `cancelled`, and `fill_rate` values
Raises	Key/type errors if a supplied report is malformed

Data types

Type	Fields
`ScheduledOrder` (frozen dataclass)	`customer_id: str`, `warehouse_id: str`, `sku: str`, `quantity: int`
`DayLog` (dataclass)	`day: int`, `delivered: list[Shipment]`, `blocked: list[Shipment]`, `fulfilled: list[Order]`, `backordered: list[Order]`, `backorders_filled: list[Order]`, `reorders: list[Shipment]`

CLI API (`scm_sim.__main__`)

Command line

Field	Content
Signature	`scm-sim [-h] [--days DAYS] [--seed SEED] [--summary] [--csv PATH]`; equivalent

Field	Content
	`python -m scm_sim ...`
Parameters	`--days`: days to simulate, integer ≥ 1, default 10. `--seed`: random seed, integer, default 42. `--summary`: print the aggregate run summary. `--csv PATH`: write daily metrics to a CSV file with the documented ten-column header. `-h`/`--help`: show usage and exit.
Returns	Exit code 0 on success; prints the daily log and final report, and optionally the run summary. If `--csv` is supplied, creates or overwrites the requested path.
Raises	`argparse` usage error on standard error with exit code 2 for invalid arguments; file-system errors from writing the requested CSV path propagate.

```
$ scm-sim --days 0
usage: scm-sim [-h] [--days DAYS] [--seed SEED] [--summary] [--csv PATH]
scm-sim: error: --days must be at least 1
```

`main`

Field	Content
Signature	`main(argv: list[str] | None = None) -> int`
Parameters	`argv`: argument list; `None` reads `sys.argv`
Returns	`0` on success
Raises	`SystemExit(2)` on an argument usage error (raised by `argparse`); CSV file errors are not converted into usage errors.

```
>>> from scm_sim.__main__ import main
>>> main(["--days", "1", "--seed", "42"])
Supply Chain Simulator demo: 1 day(s), seed 42

Day 1:
  fulfilled ORD-0001: 30 WIDGET from WH-NORTH
  fulfilled ORD-0002: 25 WIDGET from WH-SOUTH
  flags: 0 delayed, 0 stockout, 0 low, 1 excess | fill rate 1.0

=== Supply Chain Report: Day 1 ===

Delayed shipments (0)
  No issues

...
Stockouts (0)
  No issues
<BLANKLINE>
Low stock (0)
  No issues
<BLANKLINE>
Excess (1)
  - WH-NORTH / GIZMO: 650 (surplus 150)
<BLANKLINE>
Orders (2)
  pending 0, fulfilled 2, backordered 0, cancelled 0
```

```
  fill rate: 1.0
0
```

4.3.2 Order Manager API (`scm_sim.orders`)

This section documents the public interface, parameters, return types, exception behaviors, and usage examples for `OrderManager`, `Order`, and `OrderStatus`.

---

1. `OrderStatus` (Enum)

String-backed enumeration representing the lifecycle progression of customer orders. - **Values:** - `OrderStatus.PENDING`: Initial status upon order creation. - `OrderStatus.FULFILLED`: Order has been satisfied in full from inventory. - `OrderStatus.BACKORDERED`: Fulfillment attempted but deferred due to insufficient stock. - `OrderStatus.CANCELLED`: Order was revoked before fulfillment.

---

2. `Order` (Dataclass)

Data model encapsulating customer order state. - **Fields:** - `order_id: str`: Unique identifier formatted as `"ORD-NNNN"` (e.g., `"ORD-0001"`). - `customer_id: str`: Identifier for the placing customer (e.g., `"C-ACME"`). - `warehouse_id: str`: Target warehouse fulfilling the order (e.g., `"WH-NORTH"`). - `sku: str`: Item SKU requested (e.g., `"WIDGET"`). - `quantity: int`: Positive integer quantity requested. - `created_day: int`: Simulation day on which the order was created. - `status: OrderStatus`: Current lifecycle state (defaults to `OrderStatus.PENDING`). - `fulfilled_day: Optional[int]`: Day of successful fulfillment (defaults to `None`). - **Raises:** - `ValueError`: If `quantity <= 0` during initialization.

---

3. `OrderManager(inventory)`

Instantiates an `OrderManager` bound to an `Inventory` instance. - **Signature:** `OrderManager(inventory: Inventory) -> OrderManager` - **Parameters:** - `inventory` (`Inventory`): The active inventory component used for stock checks and deductions. - **Returns:** A new `OrderManager` instance. - **Raises:** None. - **Example:**

```
from scm_sim.inventory import Inventory
from scm_sim.orders import OrderManager

inv = Inventory()
orders = OrderManager(inv)
```

---

4. `create(customer_id, warehouse_id, sku, quantity, day)`

Creates, assigns a sequential identifier to, and records a new customer order. Validates that quantity > 0 prior to allocating the sequential identifier; if quantity ≤ 0, raises `ValueError` immediately without incrementing the order counter, leaving the sequence contiguous without gaps (#68). - **Signature:** `create(customer_id: str, warehouse_id: str, sku: str, quantity: int, day: int) -> Order` - **Parameters:** - `customer_id` (`str`): Unique customer identifier. - `warehouse_id` (`str`): Fulfilling warehouse identifier. - `sku` (`str`): SKU of the requested item. - `quantity` (`int`): Quantity demanded (must be > 0). - `day` (`int`): Current simulation day. - **Returns:** The newly instantiated `Order` object with status `OrderStatus.PENDING`. - **Raises:** - `ValueError`: If `quantity <= 0` (validation occurs prior to identifier allocation, preserving sequence integrity). - **Example:**

```
order = orders.create("C-ACME", "WH-NORTH", "WIDGET", 50, day=1)
assert order.order_id == "ORD-0001"
assert order.status == OrderStatus.PENDING
```

---

5. get(order_id)

Retrieves an order by its unique identifier. - **Signature:** `get(order_id: str) -> Order` - **Parameters:** - `order_id` (`str`): Identifier of the order to look up (e.g., `"ORD-0001"`). - **Returns:** The matching `Order` object. - **Raises:** - `KeyError`: If `order_id` is not tracked by the manager. - **Example:**

```
order = orders.get("ORD-0001")
print(order.status)
```

---

6. try_fulfil(order, day)

Attempts to fulfill an order immediately against warehouse inventory. - **Signature:** `try_fulfil(order: Order, day: int) -> bool` - **Parameters:** - `order` (`Order`): The order to fulfill. - `day` (`int`): Current simulation day. - **Returns:** - `True`: If stock was sufficient, stock was deducted via `inventory.remove()`, and status changed to `FULFILLED`. - `False`: If stock was insufficient (status set to `BACKORDERED`, stock untouched) or if the order was already `FULFILLED` or `CANCELLED` (state unchanged). - **Raises:** None. - **Example:**

```
success = orders.try_fulfil(order, day=1)
if not success:
    print(f"Order {order.order_id} backordered.")
```

---

7. process_open(day)

Retries fulfillment for all `PENDING` and `BACKORDERED` orders using strict oldest-first FIFO priority. Within the daily simulation cycle (`Simulation.step()`), `process_open()` is invoked prior to admitting new same-day customer demand, ensuring that existing open back-orders are served before new orders (#69). - **Signature:** `process_open(day: int) -> list[Order]` - **Parameters:** - `day` (`int`): Current simulation day. - **Returns:** A list containing all `Order` objects newly transitioned to `FULFILLED` during this retry pass. - **Raises:** None. - **Example:**

```
newly_fulfilled = orders.process_open(day=3)
for o in newly_fulfilled:
    print(f"Fulfilled {o.order_id} on day {o.fulfilled_day}")
```

---

8. cancel(order_id)

Revokes an unfulfilled order. - **Signature:** `cancel(order_id: str) -> Order` - **Parameters:** - `order_id` (`str`): Identifier of the order to cancel. - **Returns:** The cancelled `Order` object with status `OrderStatus.CANCELLED`. - **Raises:** - `ValueError`: If the order has already achieved `OrderStatus.FULFILLED`. - `KeyError`: If `order_id` is unknown. - **Example:**

```
cancelled_order = orders.cancel("ORD-0001")
assert cancelled_order.status == OrderStatus.CANCELLED
```

---

9. by_status(status)

Returns all tracked orders matching the specified lifecycle state. - **Signature:** `by_status(status: OrderStatus) -> list[Order]` - **Parameters:** - `status` (`OrderStatus`): Target status filter. - **Returns:** A list of `Order` objects whose current status matches `status`. - **Raises:** None. - **Example:**

```
backorders = orders.by_status(OrderStatus.BACKORDERED)
print(f"Total active backorders: {len(backorders)}")
```

4.3.3 Shipment Manager API

The `scm_sim.shipments` module provides classes and enumerations for managing suppliers, dispatching goods, and tracking physical shipments.

---

1. ShipmentStatus

- **Signature:** `class ShipmentStatus(str, Enum)`
- **Description:** Lifecycle state enumeration for shipments. Inherits from `str` to enable direct string comparison and JSON serialization.
- **Enumeration Members:**
  - `IN_TRANSIT = "IN_TRANSIT"`
  - `DELIVERED = "DELIVERED"`

---

2. Supplier

- **Signature:** `Supplier(supplier_id: str, name: str, lead_time_days: int = 2, reliability: float = 1.0)`
- **Parameters:**
  - `supplier_id` (*str*): Unique alphanumeric identifier for the supplier.
  - `name` (*str*): Descriptive name of the supplier company.
  - `lead_time_days` (*int*, optional): Baseline transit duration in simulated days. Default is 2. Must be `>= 0`.
  - `reliability` (*float*, optional): On-time arrival probability in range `[0.0, 1.0]`. Default is 1.0.
- **Returns:** Initialized `Supplier` dataclass instance.
- **Raises:**
  - `ValueError`: If `lead_time_days < 0` or if `reliability` is outside `[0.0, 1.0]`.
- **Example:**

```
from scm_sim.shipments import Supplier

supplier = Supplier(
    supplier_id="SUP-FLAKY",
    name="Flaky Logistics Inc",
    lead_time_days=3,
    reliability=0.75,
)
```

---

3. Shipment

- **Signature:** `Shipment(shipment_id: str, supplier_id: str, warehouse_id: str, sku: str, quantity: int, dispatch_day: int, expected_day: int, arrival_day: int, actual_day: Optional[int] = None, status: ShipmentStatus = ShipmentStatus.IN_TRANSIT)`
- **Parameters:**
  - `shipment_id` (*str*): Unique shipment tracking ID (e.g., `"SHP-0001"`).
  - `supplier_id` (*str*): Originating supplier ID.
  - `warehouse_id` (*str*): Destination warehouse ID.
  - `sku` (*str*): Stock Keeping Unit code.
  - `quantity` (*int*): Number of units shipped. Must be `> 0`.
  - `dispatch_day` (*int*): Simulation day when shipment was dispatched.
  - `expected_day` (*int*): Scheduled arrival day (`dispatch_day + lead_time_days`).
  - `arrival_day` (*int*): Actual day the shipment will arrive (`expected_day + delay`).
  - `actual_day` (*Optional[int]*, optional): Day when shipment was physically received and added to inventory. Default is `None`.
  - `status` (*ShipmentStatus*, optional): Current lifecycle status. Default is `ShipmentStatus.IN_TRANSIT`.
- **Returns:** Initialized `Shipment` dataclass instance.
- **Raises:**
  - `ValueError`: If `quantity <= 0`.
- **Example:**

```
from scm_sim.shipments import Shipment, ShipmentStatus

shp = Shipment(
    shipment_id="SHP-0001",
    supplier_id="SUP-RELIABLE",
    warehouse_id="WH-NORTH",
    sku="WIDGET",
    quantity=100,
    dispatch_day=1,
    expected_day=3,
    arrival_day=3,
)
```

---

4. Shipment.delay_days (Property)

- **Signature:** `@property def delay_days(self) -> int`
- **Parameters:** None.
- **Returns:** *int*: Number of days delayed past `expected_day` if delivered (`max(0, actual_day - expected_day)`); returns `0` if status is not `DELIVERED` or `actual_day is None`.
- **Raises:** None.
- **Example:**

```
print(shp.delay_days)  # 0 (while in transit)
```

---

5. *Shipment.days_late(current_day: int)*

- **Signature:** `days_late(current_day: int) -> int`
- **Parameters:**
  - `current_day` (*int*): Current simulation day index.
- **Returns:** *int*: If `DELIVERED`, returns `self.delay_days`. If `IN_TRANSIT`, returns `max(0, current_day - self.expected_day)`.
- **Raises:** None.
- **Example:**

```
# Dispatched day 1, expected day 3. On day 4 while in transit:
print(shp.days_late(4))  # 1
```

---

6. *ShipmentManager.__init__*

- **Signature:** `ShipmentManager(inventory: Inventory, rng: Optional[random.Random] = None)`
- **Parameters:**
  - `inventory` (*Inventory*): Reference to the shared `Inventory` system instance.
  - `rng` (*Optional[random.Random]*, optional): Seeded pseudorandom number generator. Defaults to `random.Random(42)`.
- **Returns:** Initialized `ShipmentManager` instance.
- **Raises:** None.
- **Example:**

```
import random
from scm_sim.inventory import Inventory
from scm_sim.shipments import ShipmentManager

inv = Inventory()
mgr = ShipmentManager(inventory=inv, rng=random.Random(42))
```

---

7. *ShipmentManager.add_supplier*

- **Signature:** `add_supplier(supplier: Supplier) -> None`
- **Parameters:**
  - `supplier` (*Supplier*): The supplier dataclass instance to register.
- **Returns:** None.
- **Raises:**
  - `ValueError`: If a supplier with `supplier.supplier_id` is already registered.
- **Example:**

```
mgr.add_supplier(Supplier("SUP-01", "Acme Supplier", lead_time_days=2))
```

---

8. ShipmentManager.dispatch

- **Signature:** `dispatch(supplier_id: str, warehouse_id: str, sku: str, quantity: int, day: int, delay: Optional[int] = None) -> Shipment`
- **Parameters:**
  - `supplier_id` (*str*): Registered supplier identifier.
  - `warehouse_id` (*str*): Destination warehouse identifier.
  - `sku` (*str*): Product SKU code.
  - `quantity` (*int*): Positive integer quantity to ship (`> 0`).
  - `day` (*int*): Current simulation dispatch day.
  - `delay` (*Optional[int]*, optional): Explicit delay in days. If `None`, delay is sampled via `self.rng`.
- **Returns:** *Shipment*: Newly created shipment instance with ID `SHP-0001`, `SHP-0002`, etc.
- **Raises:**
  - `KeyError`: If `supplier_id` is not registered.
  - `ValueError`: If `quantity <= 0`.
- **Example:**

```
shp = mgr.dispatch("SUP-01", "WH-NORTH", "WIDGET", quantity=50, day=1)
print(shp.shipment_id, shp.expected_day, shp.arrival_day)
```

---

9. ShipmentManager.process_arrivals

- **Signature:** `process_arrivals(day: int) -> List[Shipment]`
- **Parameters:**
  - `day` (*int*): Current simulation day index.
- **Returns:** *List[Shipment]*: All shipments delivered today whose `arrival_day <= day` and were previously `IN_TRANSIT`.
- **Raises:**
  - `CapacityExceededError`: Propagated from `Inventory.add()` if receiving units exceeds warehouse capacity.
- **Example:**

```
delivered = mgr.process_arrivals(day=3)
for s in delivered:
    print(f"Received {s.quantity} of {s.sku} at {s.warehouse_id}")
```

---

10. ShipmentManager.in_transit

- **Signature:** `in_transit() -> List[Shipment]`
- **Parameters:** None.

- **Returns:** *List[Shipment]*: Filtered list of all shipments currently in `ShipmentStatus.IN_TRANSIT`.
- **Raises:** None.
- **Example:**

```
active_shipments = mgr.in_transit()
```

---

11. ShipmentManager.delayed

- **Signature:** `delayed(current_day: int) -> List[Shipment]`
- **Parameters:**
  - `current_day` (*int*): Current simulation day index.
- **Returns:** *List[Shipment]*: List of all shipments with `days_late(current_day) > 0` (both in-transit late and delivered late).
- **Raises:** None.
- **Example:**

```
late_shipments = mgr.delayed(current_day=5)
for s in late_shipments:
    print(f"{s.shipment_id} is {s.days_late(5)} day(s) late")
```

4.4 Error Handling and Logging

Architectural Invariant: Fail Fast, Leave State Unchanged

The Supply Chain Management Simulator enforces a strict state-integrity invariant across all subsystems: **any operation that raises an exception fails immediately and leaves internal simulation state completely unchanged**.

No partial updates, corrupted records, or dangling references are permitted. If an operation fails during validation or domain constraint enforcement, the system guarantees that all in-memory collections, inventory levels, order books, and shipment logs remain identical to their pre-invocation state.

System Exceptions Reference Table

The following table details all exceptions utilized within the simulator, their triggering conditions, and their state-preservation guarantees:

Exception Class	Module Origin	Trigger Condition	State Guarantee
`ValueError`	Built-in	- Quantity `<= 0` in orders or shipments.- Supplier `lead_time_days < 0`.- Supplier `reliability` not in `[0.0, 1.0]`.- Duplicate `supplier_id` registration.- Warehouse `capacity <= 0`.- Negative `StockPolicy` parameters or `max_level < reorder_point`.-	Method exits immediately before instantiating or inserting entities. Existing inventory, orders, and suppliers remain unchanged.

Exception Class	Module Origin	Trigger Condition	State Guarantee
		Scheduled order day <= `current_day`.- Attempting to cancel an order with status `FULFILLED`.- CLI invocation with `--days < 1`.	
`KeyError`	Built-in	- Referencing an unknown `supplier_id` in `dispatch()` or `set_policy()`.- Referencing an unknown `warehouse_id` in `schedule_order()` or `add_stock()`.- Referencing a non-existent `order_id` in `get()` or `cancel()`.	No entity is created implicitly; lookup fails and dictionaries remain untouched.
`InsufficientStockError`	`scm_sim.inventory`	Calling `Inventory.remove(warehouse, sku, qty)` when `qty > level(warehouse, sku)`.	Stock level is left completely unchanged; no partial deduction occurs.
`CapacityExceededError`	`scm_sim.inventory`	Calling `Inventory.add(warehouse, sku, qty)` when adding `qty` would push total warehouse units over registered capacity.	Stock level is left completely unchanged; excess goods are rejected.
CLI Usage Error	`argparse`	Invoking the command-line interface with invalid arguments, unrecognizable flags, or non-integer values.	CLI catches the parsing error, prints standard usage text to `stderr`, and terminates with exit code 2 without starting simulation.

Error Handling Strategies across Subsystems

1. **Input and Parameter Validation:**
    - Dataclass `__post_init__` hooks (`Supplier`, `StockPolicy`, `Order`, `Shipment`) validate structural invariants at construction time.
    - Public manager methods (`OrderManager.create`, `ShipmentManager.dispatch`, `Simulation.schedule_order`) validate relational parameters before state insertion.
2. **Domain Exception Hierarchy:**
    - Custom exceptions (`InsufficientStockError`, `CapacityExceededError`) inherit from `Exception`, allowing domain callers to distinguish operational business rule rejections from systemic programming errors.
3. **Status Guarding:**
    - Order fulfillment and cancellation operations guard against invalid state regressions:
        - Orders with status `FULFILLED` cannot be cancelled (raises `ValueError`).

- Orders with status `FULFILLED` or `CANCELLED` cannot be re-processed (returns `False` safely without raising).

Logging and Observability

Because the simulator operates as an isolated in-memory Python application without external logging daemons or network sinks, observability is provided through three structured mechanisms:

1. **Structured Daily Event Logging (`DayLog`):**
   - Each simulation step creates a `DayLog` instance capturing the events of that day:
     - Arriving shipments received into inventory.
     - Customer orders created and fulfilled today.
     - Back-orders newly fulfilled during retry passes.
     - Replenishment orders dispatched to suppliers.
   - The simulation maintains a complete historical list of daily logs accessible via `Simulation.history`.
2. **Console Streaming:**
   - When executed via the CLI (`python -m scm_sim`), the runner prints a formatted execution log line for each day to `stdout`:

     ```
     Day 1: arrivals=0, orders=2, fulfilled=2, backordered=0, replenished=1
     ```

   - On completion, the CLI prints a clean, multi-line formatted summary report generated by `analytics.format_report()`.
3. **Continuous Integration Test Logging:**
   - Automated test execution through GitHub Actions captures detailed step-by-step logs for all 73 test cases with `-v` (verbose) flags and statement-by-statement coverage breakdowns via `pytest-cov`.

4.5 User Experience (UX) Design: Command-Line Interface

The Supply Chain Management Simulator exposes a command-line interface via the installed console script `scm-sim` and, for source-tree execution, `python -m scm_sim`. The user experience is designed for clarity, deterministic inspection, self-documentation, and concise reporting.

4.5.1 Command-Line Arguments and Options

The CLI uses Python's standard `argparse` library to accept operational parameters:

Argument	Type	Default	Description
`--days`	`int`	`10`	Number of simulated days to run. Must be an integer ≥ 1. Supplying a value < 1 triggers a usage error.
`--seed`	`int`	`42`	Random seed controlling supplier dispatch delays. Ensures deterministic reproducibility across runs.
`--summary`	flag	off	Prints `=== Run Summary ===` with aggregated counts for the entire simulation run.

Argument	Type	Default	Description
`--csv PATH`	path	none	Writes a CSV file with one row per day and the columns `day,delayed,stockouts,low_stock,excess,pending,fulfilled,backordered,cancelled,fill_rate`.
`-h, --help`	flag	–	Prints full usage syntax, description, and available parameter flags, then exits with code 0.

Invocation Example:

```
scm-sim --days 10 --seed 42 --summary
```

4.5.2 Day-by-Day Console Output Format

During execution, each simulation day emits a concise summary log line reflecting the core domain events:

```
Day 10:
  fulfilled ORD-0014: 15 WIDGET from WH-SOUTH
  flags: 2 delayed, 0 stockout, 0 low, 1 excess | fill rate 1.0
```

This is the actual day-10 line from the verified `scm-sim --days 10 --seed 42 --summary` run. Other days similarly report any arrivals (including capacity-blocked arrivals), fulfilled or back-ordered orders, back-orders filled and replenishment dispatches, followed by their day-level flags.

4.5.3 Final Summary Report Layout

Upon simulation completion, the CLI prints the formatted daily report for the last simulated day, and optionally the `=== Run Summary ===` block when `--summary` is supplied. Empty anomaly sections explicitly print `"No issues"` rather than remaining blank:

```
=== Supply Chain Report: Day 10 ===

Delayed shipments (2)
  - SHP-0001: GADGET to WH-SOUTH, 1 day(s) late
  - SHP-0002: WIDGET to WH-SOUTH, 1 day(s) late

Stockouts (0)
  No issues

Low stock (0)
  No issues

Excess (1)
  - WH-NORTH / GIZMO: 600 (surplus 100)

Orders (14)
  pending 0, fulfilled 14, backordered 0, cancelled 0
  fill rate: 1.0
```

The output above is from `scm-sim --days 10 --seed 42 --summary`. When `--summary` is enabled, the following verified block is printed after the final report:

```
=== Run Summary ===
  days: 10
  days_with_delays: 6
  days_with_stockouts: 3
  days_with_low_stock: 3
  days_with_excess: 10
  peak_delayed: 2
  final_fill_rate: 1.0
```

4.5.4 Process Exit Codes

The CLI conforms to standard POSIX exit status conventions: - **`0` (Success):** Simulation completed all days normally and rendered the normal day-by-day output; an optional run summary was printed if requested. - **2 (Usage Error):** Invalid command-line arguments provided (e.g., `--days 0`, unknown flags, or non-integer values). A syntax error message and usage synopsis are printed to standard error.

4.6 Open Issues and Next Steps

Resolved implementation deviations are recorded in `docs/deviations.md`. The open items below reflect the current evidence and remaining work: macOS portability has not been tested, and the final System Validation Report E1–E6 is still assigned to Rishi under issue #65. No final E1–E6 results are asserted here.

4.6.1 Known Limitations of the Current Implementation

#	Limitation	Affected component	Current behaviour	Impact	Owner
L1	**macOS portability not yet verified.** Windows 11 and Linux runs are recorded, but a macOS run is not.	Test process	SCM-NF-004 remains partially validated; the supported-runtime code uses the Python standard library.	The cross-platform claim remains incomplete until macOS is tested.	Pranav Achar
L2	**Final system validation report pending.**	Test process	Rishi is responsible for issue #65, which will deliver the final System Validation Report and its E1–E6 evidence.	Final system-level evidence remains outstanding; no E1–E6 results are inferred in this SAD.	Rishi J Mule

4.6.2 Planned Improvements

The following improvements are the intended direction for a subsequent iteration. They are not scheduled work and are not committed deliverables; they record the thinking behind the limitations above.

#	Planned improvement	Addresses	Sketch of the approach
P1	**Data-driven scenarios.** Replace the hard-coded `build_demo` scenario with CSV or JSON input.	General usability	A scenario file describing suppliers, warehouses, policies, opening stock and scheduled orders would let an analyst explore a different network without editing Python.

#	Planned improvement	Addresses	Sketch of the approach
P2	**Multi-echelon networks.** Extend the topology beyond supplier → warehouse to supplier → distribution centre → store.	General scope	A second inventory tier introduces location hierarchies, transfer orders between echelons and allocation rules on fulfilment. This is a scope increase, not a defect fix, and is out of scope for the current release.
P3	**Demand forecasting and a cost model.**	General scope	Replace the scripted order schedule with a demand model, and attach a cost per unit held, ordered and shipped so that reports can quantify the cost of a stockout or excess inventory rather than only count them.
P4	**Charted reporting.**	General scope	Render retained daily reports as time-series charts so trends are visible without post-processing. The current CLI may export daily metrics as CSV; charting is a future extension.
P5	**Continuous delivery with a release package.** Extend the CI pipeline beyond the `test` job.	General process	A tagged-release build job could assemble a distributable artifact in addition to the current test and coverage reporting workflow.
P6	**Richer temporal analytics.**	Run-level analytics	`summarize_run` reports day counts and peak/final values; it does not calculate trend series, average delay, or time-to-resolution. A future analytics extension could compute these measures from retained daily history and shipment lifecycle timestamps.

4.6.3 Immediate Next Steps

1. Complete issue #65 by preparing the final System Validation Report with its E1–E6 evidence; Rishi owns that report. This SAD does not pre-empt or infer its results.
2. Run and record the outstanding macOS portability test for SCM-NF-004 before claiming full cross-platform validation.

5. Appendices

5.1 Glossary

Term	Definition
Back-order	A customer order that could not be fulfilled immediately due to insufficient on-hand inventory and remains open in the backlog to be retried on subsequent simulation days.
CI (Continuous Integration)	Automated pipeline (implemented via GitHub Actions) that builds, tests, and validates code changes on every pull request and push to `main`.
Days Late	The number of days a shipment is delayed beyond its scheduled `expected_day`. If delivered, evaluated as `actual_day - expected_day`; if in transit, evaluated as `max(0, current_day - expected_day)`.
Excess Inventory	Units of stock held in a warehouse that exceed the configured maximum threshold (`max_level`) for a specific SKU.
FIFO (First-In, First-Out)	Allocation priority rule ensuring open back-orders with the oldest creation day (`created_day`) are fulfilled first during inventory replenishment passes.
Fill Rate	Operational service level metric computed as $\frac{\text{Fulfilled Orders}}{\text{Total Orders} - \text{Cancelled Orders}}$, rounded to two decimal places.
Lead Time	The standard transit duration in simulated days required for goods to travel from a supplier to a destination warehouse.
PRNG	Pseudorandom Number Generator. Uses an injected `random.Random(seed)` instance to achieve deterministic stochastic simulations.
Reliability	The probability ($p \in [0.0, 1.0]$) that a supplier delivers a dispatched shipment on time without transit delay.
Reorder Point	Inventory threshold level at or below which an automated replenishment reorder is triggered.
Reorder Quantity	Fixed batch size of units ordered from a supplier when replenishment is initiated.
RTM	Requirements Traceability Matrix; bidirectional mapping connecting requirements to architecture modules and test cases.
SKU	Stock Keeping Unit; unique alphanumeric identifier representing a distinct product item.
Stockout	Operational condition where available warehouse inventory for an active SKU is exactly zero (`level == 0`).

5.2 References

1. **IEEE Std 42010-2011:** *Systems and software engineering — Architecture description*. Institute of Electrical and Electronics Engineers.
2. **IEEE Std 29148-2018 (superseding IEEE Std 830-1998):** *Systems and software engineering — Life cycle processes — Requirements engineering*.

3. **IEEE Std 829-2008:** *Standard for Software and System Test Documentation*.
4. **Python Software Foundation:** *The Python Standard Library Documentation (Version 3.11)*. Available at: https://docs.python.org/3.11/
5. **pytest Documentation:** *pytest: helps you write better programs (Version 8.0)*. Available at: https://docs.pytest.org/
6. **GitHub Actions Documentation:** *Automating your workflow with GitHub Actions*. Available at: https://docs.github.com/en/actions
7. **Open Web Application Security Project (OWASP):** *Top 10 CI/CD Security Risks*. Available at: https://owasp.org/www-project-cicd-top-10/

5.3 Tools and Environment

Tool / Technology	Version / Platform	Architectural Purpose
Python	3.11.9 (64-bit)	Primary development language and runtime execution environment.
pytest	9.1.1 (>= 8.0)	Automated testing framework for executing unit, integration, and system test suites.
pytest-cov	7.1.0 (>= 5.0)	Code coverage measurement tool producing statement coverage reports.
Git	2.51.1	Distributed version control tracking commits, branches, and development history.
GitHub	Cloud Platform	Remote repository hosting, branch protection enforcement, code reviews, and issue tracking.
GitHub Actions	Hosted `ubuntu-latest`	Continuous Integration runner executing automated test verification on every pull request.
PlantUML	Standard Syntax (planttext.com)	Unified Modeling Language (UML) diagram generation for sequence and component models.
Visual Studio Code / Antigravity IDE	Modern IDE	Development environment, linting, terminal execution, and markdown editing.